11 ROZDZIAŁÓW · 172 PYTAŃ PROFESORA · 3 DIAGRAMY

Stanowisko
Projekt wyjaśniony przez profesora

Od pomysłu i zależności po polityki Androida, PIN, WebView i zachowanie po awarii. Pytania, konkretne odpowiedzi i odnośniki do implementacji.

Wersja kodu: 1.0.9 (11) · gałąź android · analiza i weryfikacja: 6 listopada 2026

Jak korzystać z przewodnika

Stanowisko zamienia urządzenie z Androidem w kontrolowany pulpit do uruchamiania wybranych aplikacji i strony internetowej. Ograniczenia egzekwuje system Android przez Device Owner i Lock Task. PIN pozwala administratorowi zmieniać konfigurację i kończyć sesje jednej aplikacji.

To podręcznik do kodu w `/home/atur/Desktop/MYAPP`, gałąź `android`, commit `bb7dd3c`, wersja aplikacji `1.0.9` (`versionCode = 11`). Stan opisano i sprawdzono 6 listopada 2026 r. Rozmowę prowadzi profesor, który sam zadaje pytania, rozbiera mechanizmy na części i odpowiada także na pytanie „dlaczego tak”.

Czytaj w przeglądarce: pełny przewodnik HTML. Działa lokalnie, bez pobierania bibliotek, ma spis treści, wyszukiwanie rozdziałów i widok do druku. Pliki poniżej są edytowalnym źródłem dokumentacji.

Do czytania i drukowania: pełny przewodnik PDF.

Kolejność wykładów

Rozdział	Co wyjaśnia
1. Cel i pochodzenie	Co to za aplikacja, skąd pochodzi kod, dla kogo ma sens i czego nie obejmuje
2. Architektura i pliki	Wszystkie własne pliki Kotlin, zasoby, konfiguracja i zależności między częściami
3. Dane i stan	Room, migracje, SharedPreferences, Flow, rewizje i spójność
4. Egzekwowanie polityki	Zapis, zastosowanie, sesje, przywracanie po restarcie i wyjście serwisowe
5. PIN i uprawnienia	Kryptografia, autoryzacja, czas dostępu administratora i decyzje o uprawnieniach aplikacji
6. Pulpit i strona	Compose, cykl życia, pływające menu, WebView i granice filtrowania adresów
7. Budowanie i instalacja	Wersje narzędzi, zależności Gradle, APK, podpisy, Linux i Windows
8. Decyzje i ograniczenia	Dlaczego takie podejścia, realne alternatywy, ryzyka i kierunki rozwoju
9. Obsługa i nauka	Procedury, diagnoza problemów, scenariusze testów, ćwiczenia i pytania na obronę
10. Źródła i weryfikacja	Pochodzenie twierdzeń, wyniki kontroli, słownik i aktualizowanie podręcznika
11. Scenariusz prezentacji	Co mówić do wszystkich 47 slajdów Adrian_Tur_1.pptx, krótkie warianty, przejścia, demonstracja i pytania profesora

Do wystąpienia korzystaj z osobnego scenariusza HTML, scenariusza PDF, krótkiej ściągi PDF lub kopii prezentacji z notatkami prelegenta.

Trzy zasady czytania

Fakt z implementacji oznacza zachowanie wynikające z konkretnego kodu. Odnośniki prowadzą do plików w tym repozytorium.

Uzasadnienie inżynierskie wyjaśnia sens rozwiązania i jego kompromisy. Jeśli historia nie zapisuje motywacji autora, nie przypisujemy mu takich intencji. Rozdział 8 porównuje implementację z alternatywami właśnie w tym znaczeniu.

Wynik sprawdzenia dotyczy tylko wskazanej próby. Udana kompilacja nie dowodzi szczelności kiosku na rzeczywistym tablecie. Stan kontroli i zakres wymagający urządzenia znajdują się w rozdziale 10.

Ścieżki dla różnych czytelników

- **Chcę zrozumieć pomysł:** rozdziały 1, 4 i 8.
- **Chcę czytać i zmieniać kod:** rozdziały 2–7, potem ćwiczenia w 9.
- **Chcę uruchomić tablet:** rozdziały 7 i 9, z uwzględnieniem granic z 8.
- **Chcę przedstawić projekt na zajęciach:** rozdział 11 dopasowany do prezentacji, następnie rozdział 1, diagramy, przykład rewizji w 3 i pytania na obronę w 9.

Najważniejsza zależność całego projektu brzmi: **instalacja APK → nadanie Device Owner → przygotowanie**

uprawnień i nakładki → zapis konfiguracji → zastosowanie polityki → chroniona praca. Każda strzałka oznacza osobny krok, z osobnymi warunkami i możliwymi błędami.

1. Cel i pochodzenie projektu

Problem rozwiązywany przez Stanowisko

Profesor pyta: co właściwie otrzymaliśmy w katalogu MYAPP?

Otrzymaliśmy natywną aplikację Android o nazwie **Stanowisko**, napisaną w Kotlinie. Jej identyfikatorem instalacyjnym jest `pl.adriantur.stanowisko`. Nazwa katalogu `MYAPP` jest lokalną nazwą folderu; nie wyznacza nazwy pakietu, produktu ani modułu Gradle. Projekt ma jeden moduł aplikacji, `:app`.

Aplikacja łączy trzy odpowiedzialności: własny pulpit urządzenia, lokalny panel administratora i sterownik polityk urządzenia. Pokazuje dozwolone aplikacje oraz opcjonalny kafelek strony internetowej. Kiedy zostaje odpowiednio przygotowana i uruchomi ochronę, korzysta z Androida do ograniczenia dostępnych zadań.

Źródła: konfiguracja aplikacji, manifest, PolicyEngine.kt.

Profesor pyta: jaki praktyczny problem uzasadnia taką konstrukcję?

Zwykły tablet daje użytkownikowi dostęp do wielu aplikacji, ustawień i systemowego pulpitu. Stanowisko pozwala przygotować urządzenie do konkretnego zadania: prezentacji strony, pracy z wybranym programem albo przełączania się między kilkoma dozwolonymi narzędziami. Przykłady zastosowań to kiosk informacyjny, stanowisko laboratoryjne czy tablet pokazujący panel organizacji. Są to przykłady wynikające z możliwości aplikacji, a nie zaszyte w kodzie branżowe przeznaczenie.

Administrator wybiera pakiety i tryby pracy, a użytkownik końcowy korzysta z przygotowanego pulpitu. Administrator i użytkownik nie są tu kontami w bazie. Rozróżnia ich chwilowa autoryzacja PIN-em wewnątrz tej samej aplikacji i tego samego użytkownika Androida.

Profesor pyta: czy wystarczy narysować pulpit i schować przycisk Wstecz?

Nie. Sam interfejs może utrudnić wyjście, ale uprawnienia do uruchamiania zadań nadal należą do systemu. Stanowisko ustawia listę pakietów przez `DevicePolicyManager`, a następnie sprawdza, czy urządzenie weszło w `LOCK_TASK_MODE_LOCKED`. Android odróżnia ten stan od `LOCK_TASK_MODE_PINNED`, czyli przypięcia ekranu. Te mechanizmy mają inne warunki opuszczenia. Opis platformy: Lock task mode.

Zakres funkcji

Profesor pyta: co administrator naprawdę może skonfigurować?

Obszar	Zachowanie obecnej implementacji
Lista aplikacji	Wybór spośród pakietów posiadających aktywność LAUNCHER; Stanowisko dodaje własny pakiet automatycznie do polityki

Obszar	Zachowanie obecnej implementacji
Tryb aplikacji	Osobno dla każdego pakietu: wiele aplikacji albo sesja jednej aplikacji
Uprawnienia	Przyznanie lub jawna odmowa zadeklarowanych uprawnień typu dangerous, w granicach dopuszczonych przez Androida
Strona	Tytuł, URL, dodatkowe dozwolone origins, JavaScript, tryb jednej lub wielu aplikacji
Ekran	Niewygaszanie i ukrywanie pasków systemowych podczas ochrony
Administracja	PIN, wyjście serwisowe, restart urządzenia i podgląd lokalnego dziennika
Sterowanie nad aplikacjami	Przesuwane menu powrotu do pulpitu i autoryzacji administratora

Źródła: WorkspaceScreen.kt, WebsiteSettings.kt, AppPermissions.kt.

Profesor pyta: czym różnią się dwa tryby pracy?

W trybie wielu aplikacji użytkownik może wracać do pulpitu Stanowiska i otwierać inne dozwolone aplikacje. Stan chronionej pracy pozostaje aktywny. Gdy paski są ukryte, służy do tego ikona domu w nakładce.

Sesja jednej aplikacji dodatkowo zawęża politykę do własnego pakietu Stanowiska oraz wybranej aplikacji. Dla strony internetowej pozostaje tylko własny pakiet, bo `WebActivity` jest jego częścią. W preferences powstaje znacznik sesji. Zwykły powrót do pulpitu wymaga wtedy PIN-u; samo otwarcie `MainActivity` prowadzi do próby odtworzenia aktywnej sesji.

To rozróżnienie działa **dla poszczególnych aplikacji**, a nie jako jeden globalny przełącznik wszystkich kafelków. Na jednym pulpicie mogą leżeć aplikacja wielozadaniowa i aplikacja uruchamiająca sesję pojedynczą.

Pochodzenie i historia

Profesor pyta: skąd pochodzi projekt i czy jest aplikacją webową?

Lokalne metadane Git wskazują `origin` na `https://github.com/Shreider/tablet_app_web.git`, a bieżącą gałęzią jest `android`. README rozróżnia gałąź `android` z aplikacją natywną i `main` z aplikacją webową. W tym katalogu analizujemy wyłącznie zawartość gałęzi Android; nie ma tutaj źródeł backendu ani frontendu tamtej aplikacji.

WebView może wyświetlić usługę dostępną pod skonfigurowanym adresem, ale samo Stanowisko nie definiuje jej API, modelu danych ani logowania. Nie znaleziono w tym drzewie klienta Retrofit, serwera HTTP, kont użytkowników, synchronizacji z centralą ani połączenia z bazą w chmurze. Uwierzytelnienie strony pozostaje sprawą strony i jej serwera.

Źródło deklaracji podziału: README projektu. Pochodzenie lokalnej kopii można odtworzyć poleceniami `git remote -v` i `git branch --show-current`.

Profesor pyta: co historia pozwala powiedzieć o rozwoju?

Lokalna historia zawiera pięć commitów, wszystkie z 10 października 2026 r.:

Commit	Treść zmiany wynikająca z historii
`47cc5b3`	Początkowy import Stanowiska 1.0.9, konfiguracji i instalatora Linux
`966e240`	Zachowanie wejścia skryptu podczas odczytów ADB
`ee599d1`	Przygotowanie roli właściciela urządzenia w instalatorze
`54a0e02`	Instalator Windows z konfiguracją właściciela
`bb7dd3c`	Obsługa znacznika UTF-8 BOM w komendzie instalacji Windows

Pierwszy commit już zawiera rozwiniętą aplikację i schematy Room 1, 2 oraz 3. To ważne: dostępna historia nie pokazuje całej drogi od pierwszego prototypu. Numerów starszych wersji produktu, autorstwa poszczególnych pomysłów ani powodów wyboru wszystkich bibliotek nie da się uczciwie odtworzyć z samego numeru `1.0.9`.

Profesor pyta: skąd wiadomo, dlaczego zastosowano konkretne podejście?

Część przyczyn jest zapisana w komentarzach. Przykładowo silnik zapisuje poprzedni stan launcherów przed ich ukryciem, aby móc je przywrócić również po restarcie. `serviceExit()` najpierw utrwala wyłączenie polityki, żeby ponowne uruchomienie nie przywróciło ograniczeń. `PinStore` ustawia PIN domyślny tylko przy braku poświadczenia.

Dla innych wyborów przedstawiamy **uzasadnienie inżynierskie**: jakie właściwości daje Compose, dlaczego Room pomaga przy migracjach, jaki koszt ma brak rozdzielonych modułów. Taka analiza tłumaczy sens rozwiązania, ale nie udaje relacji z rozmowy z autorem. Porównania znajdują się w rozdziale 8.

Wymagania i granice produktu

Profesor pyta: czego potrzebuje uruchomienie?

Aplikacja ma `minSdk = 34`, czyli Android 14, i celuje w API 36. Pełna ochrona wymaga roli Device Owner, zezwolenia na wyświetlanie nad innymi aplikacjami, zapisanej polityki i przygotowanych uprawnień wybranych programów. Strona wymaga ponadto działającego systemowego WebView i dostępu do swojego serwera. Sam lokalny pulpit i jego baza nie potrzebują serwera Stanowiska.

Profesor pyta: co oznacza „lokalna aplikacja”, skoro używa internetu?

Logika konfiguracji, PIN i dziennik są lokalne. Internet jest potrzebny do pobrania zależności podczas pierwszego budowania, pobrania APK przez instalator i otwierania skonfigurowanej strony. Dostępność konkretnej strony bez sieci zależy od niej i zachowania WebView; projekt nie implementuje własnego mechanizmu offline dla treści WWW.

Profesor pyta: jakie zdanie najlepiej przedstawia projekt na obronie?

„Stanowisko to lokalnie zarządzany launcher i kontroler polityk Androida, który udostępnia wybrane aplikacje oraz stronę WWW, rozdziela zapis konfiguracji od jej zastosowania i używa systemowego Lock Task do ograniczania pracy urządzenia”.

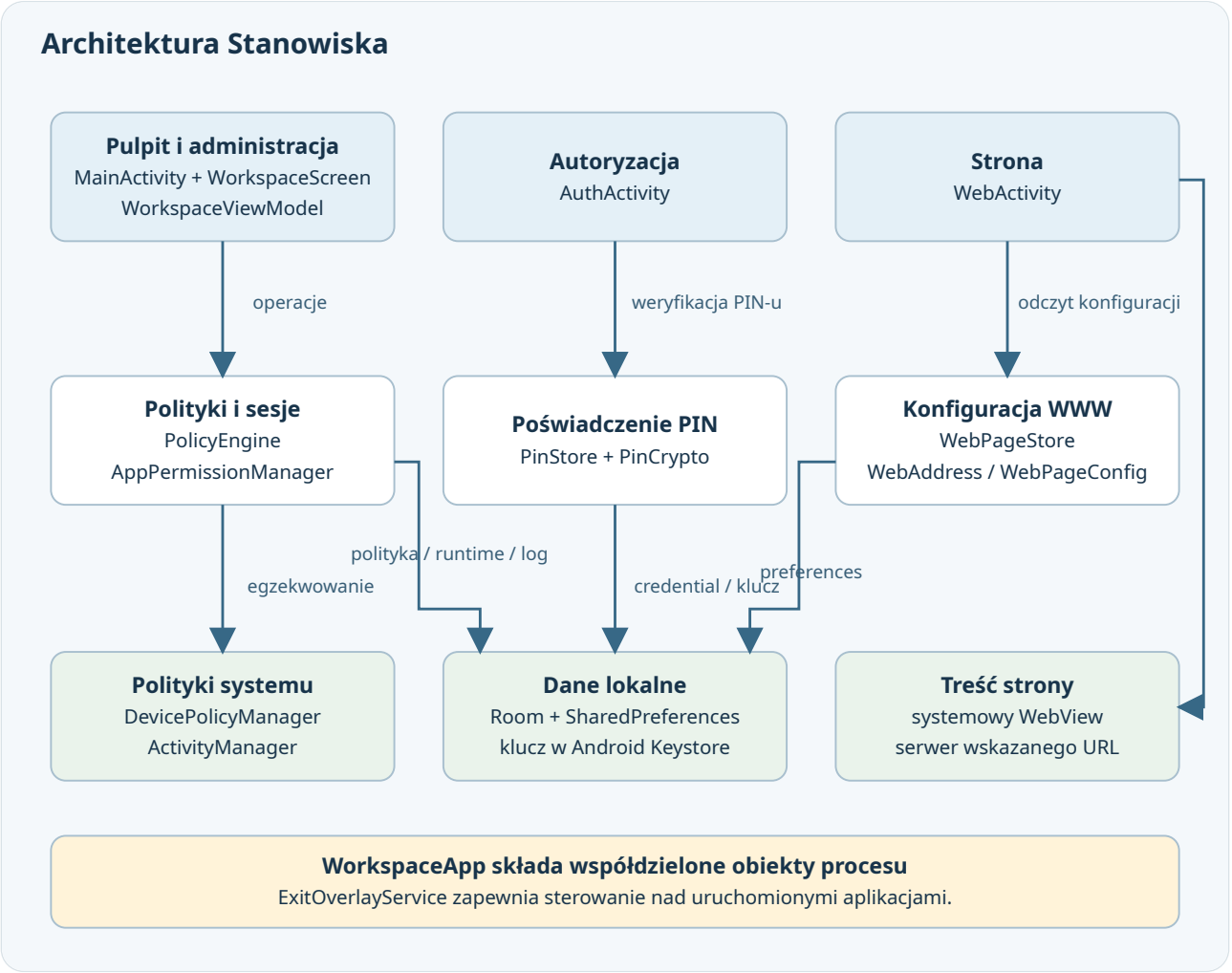
To zdanie nazywa odpowiedzialności i mechanizm. Pozostałe rozdziały pokazują dokładnie, gdzie i pod jakimi warunkami każda jego część jest prawdziwa.

2. Architektura i mapa plików

Podział odpowiedzialności

Profesor pyta: jak podzielić aplikację w głowie, zanim zaczniemy czytać klasy?

Na warstwę prezentacji, operacje dotyczące urządzenia oraz przechowywanie stanu. Podział jest logiczny; wszystkie klasy należą do jednego pakietu i jednego modułu Gradle. Kod przypomina MVVM, bo ekran obserwuje `WorkspaceViewModel`, lecz część działań ekran wywołuje bezpośrednio na usługach `WorkspaceApp`. Nie jest to konsekwentnie odizolowana Clean Architecture.



Zależności głównych części Stanowiska

`WorkspaceApp` tworzy i udostępnia obiekty współdzielone w procesie. `WorkspaceViewModel` przechowuje stan prezentacji, obserwuje Room i opakowuje operacje w obsługę zajętości oraz błędów. `PolicyEngine` tłumaczy żądania na wywołania systemowego `DevicePolicyManager`. `PinStore` i `AppPermissionManager` obsługują własne obszary. Room i SharedPreferences przechowują dane lokalne.

Źródła: WorkspaceApp.kt, WorkspaceViewModel.kt, PolicyEngine.kt.

Profesor pyta: dlaczego zależności spotykają się w klasie Application?

Android tworzy `WorkspaceApp` dla procesu aplikacji. Jej właściwości `by lazy` pozwalają uzyskać jedną instancję bazy, silnika, magazynu PIN-u i konfiguracji strony w tym procesie. Jest to ręczne składanie zależności. Nie ma Hilt, Dagger ani kontenera DI.

Zyskiem jest mała liczba elementów konfiguracji. Kosztem jest silne związanie klas z `WorkspaceApp` i Androidem. Na przykład testowanie `PolicyEngine` bez platformy wymagałoby wydzielenia interfejsu systemowych polityk. Samo `by lazy` nie zapewnia trwałości obiektu po zabiciu procesu.

Trzy aktywności i nakładka

Profesor pyta: dlaczego są trzy Activity?

`MainActivity` jest pulpitem i gospodarzem ekranu administracyjnego. `AuthActivity` to oddzielny ekran autoryzacji dostępny z nakładki. `WebActivity` utrzymuje WebView we własnym zadaniu. Dzięki temu strona, pulpit i okno PIN-u mogą mieć osobne zachowanie powrotu i usuwania zadania.

Komponent	Deklaracja i znaczenie
`MainActivity`	`exported=true`, `singleTask`, kategorie HOME i LAUNCHER; może być pulpitem Androida i zwykłą ikoną startową
`AuthActivity`	`exported=false`, `singleTask`, affinity `.auth`, wykluczenie z ostatnich zadań
`WebActivity`	`exported=false`, `singleTask`, affinity `.web`, wykluczenie z ostatnich zadań
`ExitOverlayService`	Prywatny foreground service typu `specialUse`, tworzący okno nad aplikacjami
`WorkspaceAdminReceiver`	Odbiornik administracyjny chroniony `BIND_DEVICE_ADMIN`
`BootReceiver`	Prywatny odbiornik uruchomienia systemu i aktualizacji własnego pakietu

Źródło: AndroidManifest.xml. Affinity określa powiązanie aktywności z zadaniami; nie tworzy osobnego procesu ani osobnego użytkownika Androida.

Profesor pyta: które wejścia są dostępne innym aplikacjom?

Manifest eksportuje pulpit i odbiornik administracyjny. Ten drugi wymaga systemowego uprawnienia `BIND_DEVICE_ADMIN`. Ekrany PIN-u i strony oraz usługa nakładki nie są eksportowane. Uruchomienie publicznego pulpitu nie nadaje autoryzacji administratora: wrażliwe metody sprawdzają `requireAdmin()` lub wymagają stanu polityki. Nie oznacza to automatycznego dowodu odporności wszystkich przepływów; warunki każdej operacji są omówione w rozdziałach 4 i 5.

Wszystkie pliki Kotlin

Profesor pyta: gdzie szukać konkretnej funkcji?

Poniższa mapa obejmuje wszystkie 26 plików Kotlin aplikacji. W tabeli nazwa pliku prowadzi bezpośrednio do źródła.

Plik	Odpowiedzialność i najważniejsze wejścia
WorkspaceApp.kt	Składanie usług, inicjalizacja PIN-u, locale, cykl życia Activity, `authorized`, `requireAdmin`, `audit`
MainActivity.kt	Tworzenie ViewModelu, oczekiwanie na inicjalizację, `onResume` z `restore`, przekazywanie żądania uprawnień
WorkspaceViewModel.kt	Strumienie polityki i logów, lista aplikacji, liczniki uprawnień, `refresh`, `run`, `notify`
WorkspaceScreen.kt	Panel i wybór widoku, formularz listy, dialog uprawnień, zapis i zastosowanie, zmiana PIN-u, serwis
VisualDesign.kt	Motyw, tło, kafelki launchera, zegar, bateria, ikony rysowane w Canvas, komunikaty
PolicyEngine.kt	Lista programów, egzekwowanie i przywracanie polityki, uruchamianie sesji, zakończenie, serwis
AppPolicy.kt	`AppRule`, kodowanie JSON, walidacja listy dozwolonych aplikacji
WorkspaceDatabase.kt	Encje Room, DAO, baza wersji 3 i dwie migracje
AuthActivity.kt	Sprawdzenie PIN-u, autoryzowany powrót, zakończenie sesji, przejście do uprawnień
AdministratorPinField.kt	Pole cyfr z maskowaniem, klawiaturą PIN i limitem 12 znaków
PinStore.kt	AES-GCM z Android Keystore, zapis credential, weryfikacja i liczniki prób
PinCrypto.kt	Format PIN-u, PBKDF2 i obliczanie czasu blokady
AppPermissions.kt	Model uprawnienia, walidacja zależności lokalizacji, kolejność zmian, weryfikacja i próba wycofania
ExitOverlayService.kt	Powiadomienie usługi, okno, przeciąganie, przyciski, potwierdzenie gotowości
FloatingPanelPosition.kt	Obliczenia granic, przycinanie pozycji i zapis współrzędnych jako ułamków
WebPageConfig.kt	Konfiguracja strony, normalizacja origin, walidacja adresów i magazyn preferences z StateFlow
WebsiteSettings.kt	Formularz strony; przełącznik włączenia zapisuje od razu, inne pola przez przycisk
WebActivity.kt	WebView, kontrola nawigacji, odmowy dostępu do plików i urządzeń, obsługa awarii renderera
BootReceiver.kt	Żądanie odtworzenia po BOOT_COMPLETED i MY_PACKAGE_REPLACED
WorkspaceAdminReceiver.kt	Rejestrowanie systemowego wejścia i wyjścia z Lock Task
WindowProtection.kt	Pomocnicze ustawienie flagi niewygaszania oraz widoczności pasków okna
BatteryStatus.kt	Odczyt procentu i ładowania, odbiornik związany z cyklem życia Compose
LenovoSystemMenu.kt	Rozpoznanie menu producenta na konkretnym TB311FU; tylko odczyt ustawienia
UiText.kt	Polskie nazwy stanów Lock Task oraz prezentacja zdarzeń audytowych
WorkspaceError.kt	Własny wyjątek, sprawdzanie warunków, bezpieczny komunikat dla użytkownika
WorkspaceNotice.kt	Typ komunikatu i zasada jego wygaszania

Skąd biorą się klasy i wywołania

Profesor pyta: gdzie jest funkcja main uruchamiająca całą aplikację?

Własny kod nie rozpoczyna się tutaj od zwykłego `fun main()`. Android odczytuje manifest, tworzy wskazaną klasę `WorkspaceApp` i uruchamia odpowiedni komponent. Dotknięcie ikony kieruje do `MainActivity` dzięki filtrowi LAUNCHER, wybór pulpitu korzysta z HOME, a zdarzenie startu systemu trafia do odbiornika. Metody `onCreate`, `onResume`, `onReceive` i `onStartCommand` są punktami wywoływanymi przez platformę. Dlatego do zrozumienia kolejności trzeba czytać manifest i cykl życia, a nie szukać jednego liniowego programu.

Profesor pyta: skąd pochodzą importy, skoro wielu klas nie ma w repozytorium?

`android.*` to API platformy udostępnione do kompilacji przez SDK, a realizowane na urządzeniu przez Androida. `androidx.*` pochodzi z bibliotek AndroidX zadeklarowanych bezpośrednio lub przechodnio w Gradle. `kotlinx.coroutines.*` pochodzi z korutyn, `java.*` i `javax.crypto.*` z odpowiednich API biblioteki standardowej i platformy. `pl.adriantur.stanowisko.*` to własny pakiet projektu. Sam `import` jedynie pozwala używać krótszej nazwy klasy; nie pobiera i nie instaluje zależności. Pobieranie rozstrzyga Gradle, a dostępność platformowych API także minSdk i compileSdk.

Profesor pyta: dlaczego używamy R.string, choć nie ma pliku R.kt?

Klasa `R` jest generowana z zasobów w `res`. Nazwa `R.string.error_operation` identyfikuje tekst w `strings.xml`, a `getString(...)` odczytuje go dla kontekstu i konfiguracji językowej. Analogicznie `R.drawable` identyfikuje ikony. Tych identyfikatorów nie wpisujemy ręcznie jako stałych liczbowych. Room również tworzy kod, ale przez KSP i na podstawie encji oraz DAO. W obu przypadkach autor utrzymuje źródło deklaracji, a generator buduje potrzebną implementację lub identyfikatory.

Profesor pyta: co oznaczają by lazy, remember i suspend w tym projekcie?

`by lazy` tworzy wartość przy pierwszym użyciu i zapamiętuje ją w danej instancji obiektu; tak `WorkspaceApp` udostępnia usługi. `by viewModels` uzyskuje ViewModel związany z właścicielem Androida zamiast tworzyć nowy obiekt przy każdym przerysowaniu. `remember` zachowuje stan w żyjącej kompozycji UI, ale nie jest zapisem na dysku. To trzy różne zakresy życia.

`suspend` oznacza funkcję zdolną do zawieszenia pracy korutyny; nie gwarantuje samoistnego wykonania na wątku tła. Dopiero `withContext(Dispatchers.IO)` przełącza odpowiedni fragment na pulę dla IO, a `delay` zawiesza oczekiwanie. `LaunchedEffect` uruchamia zadanie związane z życiem elementu Compose i jego kluczami, `DisposableEffect` pozwala sprzątnąć zewnętrzny efekt, a `collectAsStateWithLifecycle` łączy strumień ze stanem ekranu. Te konstrukcje tłumaczą, dlaczego dane mogą się zmieniać bez ręcznego przerysowywania całej listy.

Pliki poza Kotlinem

Profesor pyta: co jeszcze bierze udział w działaniu?

Plik lub katalog	Rola

Plik lub katalog	Rola
settings.gradle.kts	Nazwa `Stanowisko`, moduł `:app`, repozytoria pluginów i bibliotek
build.gradle.kts	Wersje pluginów AGP, Compose compiler i KSP
app/build.gradle.kts	API, wersja produktu, Java/Kotlin target, Compose i biblioteki
gradle.properties	Maksymalna sterta Gradle 3 GB, kodowanie UTF-8 i AndroidX
gradle/wrapper	Wrapper Gradle 9.1.0 z sumą archiwum dystrybucji
gradlew, gradlew.bat	Start właściwej wersji Gradle na Unix i Windows
app/schemas	Eksporty kontraktu bazy wersji 1–3; nie są bazą z danymi użytkownika
manifest debug	`testOnly=true` dla wariantu rozwojowego
strings.xml	Polskie teksty, komunikaty, opisy dostępności i nazwa aplikacji
styles.xml	Bazowy motyw okien Androida, niezależny od funkcji motywu Compose
device_admin.xml	Metadane odbiornika administracyjnego; pusty `uses-policies` nie nadaje Device Owner
locale_config.xml	Zadeklarowany język polski
data_extraction_rules.xml	Wyłączenie eksportu danych w backupie i transferze urządzenia
`res/drawable/ic_*.xml`	Pięć ikon wektorowych: aplikacja, administrator, przesuwanie, dom i kłódka
install.sh, install.ps1	Pobieranie gotowego APK, kontrola sumy, ADB i opcjonalne przygotowanie właściciela
.gitignore	Wykluczenie cache, buildów, ustawień lokalnych, kluczy i APK
.editorconfig, .gitattributes	Kodowanie, wcięcia i zakończenia linii; JAR jest plikiem binarnym

`local.properties` wskazuje lokalny SDK i jest ignorowany przez Git. `.idea`, `.gradle`, `.kotlin` oraz `app/build` zawierają stan środowiska lub wyniki narzędzi. Nie należy ich traktować jako osobnych warstw architektury ani ręcznie utrzymywanego kodu aplikacji.

Profesor pyta: jak czytać kod w sensownej kolejności?

Najpierw manifest, następnie `WorkspaceApp`, `MainActivity`, `WorkspaceViewModel` i `AppPolicy`. Potem model bazy i najważniejsze metody `PolicyEngine`: `save`, `apply`, `launch`, `restore`, `endSession`, `serviceExit`. Dopiero wtedy ekran administracyjny, PIN, uprawnienia i WebView. W tej kolejności znamy już znaczenie obiektów, na które natrafimy w dużych plikach UI.

Przy zmianie jednej funkcji idziemy po konkretnej ścieżce. Przykładowo nowe ustawienie ekranu wymaga przemyślenia formularza, zapisu, kopii zastosowanej, migracji, runtime, przywracania i serwisu. Sama nowa kontrolka nie kończy implementacji.

3. Dane i stan aplikacji

Cztery miejsca przechowywania

Profesor pyta: gdzie naprawdę znajduje się stan Stanowiska?

W czterech rodzajach miejsc: baza Room, prywatne SharedPreferences, pamięć procesu i stan systemu Android. To rozróżnienie tłumaczy wiele pozornych sprzeczności, np. zapis „polityka aktywna” przy błędzie jej odtworzenia.

Miejsce	Przykład	Co przetrwa utratę procesu
Room `workspace.db`	Zapisana i zastosowana polityka, dziennik	Dane zapisane w bazie
SharedPreferences	`runtime.session`, credential PIN-u, ustawienia WWW	Zatwierdzone wartości preferences
Pamięć procesu	`adminUntil`, `busy`, `lockRootActivity`, formularze `remember`	Te wartości same z siebie nie przetrwają
Android	Rola Device Owner, lista Lock Task, wybór HOME, przyznane uprawnienia	Zależnie od konkretnej polityki i mechanizmu systemowego; odtwarzanie musi odczytać rzeczywisty stan

Źródła: WorkspaceDatabase.kt, WorkspaceApp.kt, PolicyEngine.kt.

Model polityki w Room

Profesor pyta: co zawiera PolicyRecord?

Jeden rekord o domyślnym `id = 1`. DAO odczytuje właśnie ten identyfikator. Baza nie modeluje wielu nazwanych profili konfiguracji.

Pole	Znaczenie
`id`	Klucz główny; konwencja aplikacji używa 1
`revision`	Numer ostatniego zapisu konfiguracji; start 0
`rules`	JSON reguł zapisanych przez administratora
`appliedRevision`	Numer rewizji, której zastosowanie zakończono sukcesem; start −1
`appliedRules`	Kopia listy z ostatniego udanego zastosowania
`enabled`	Czy aplikacja ma odtwarzać ochronę
`error`	Zachowany opis ostatniego problemu zastosowania lub odtworzenia
`keepScreenOn`	Zapisana decyzja o niewygaszaniu

Pole	Znaczenie
`appliedKeepScreenOn`	Zastosowana decyzja o niewygaszaniu
`hideSystemBars`	Zapisana decyzja o paskach
`appliedHideSystemBars`	Zastosowana decyzja o paskach

Profesor pyta: po co dwie listy i dwa numery rewizji?

Rozważmy konkretny przebieg:

1. Rewizja 7 pozwala na aparat i zegar. Została zastosowana, więc `revision = appliedRevision = 7`.
2. Administrator usuwa aparat i zapisuje konfigurację. Powstaje rewizja 8, lecz `appliedRevision` nadal wynosi 7.
3. Gdy `enabled = true`, pulpit i `launch()` używają `appliedRules`. Aparat nadal należy do aktywnej konfiguracji.
4. Dopiero udane `apply()` kopiuje nowe reguły do `appliedRules` i ustawia `appliedRevision = 8`.
5. Jeśli zastosowanie zawiedzie, aplikacja próbuje przywrócić poprzednią politykę i zachowuje błąd. Sam zapis rewizji 8 nie staje się dowodem jej aktywacji.

Ta konstrukcja oddziela życzenie administratora od ostatniej potwierdzonej konfiguracji. `appliedRevision` jest zapisem wyniku operacji aplikacji, a nie ciągłym dowodem, że każda polityka Androida nadal ma dokładnie ten stan.

Profesor pyta: jak wygląda AppRule i skąd bierze się JSON?

```
data class AppRule(val packageName: String, val singleSession: Boolean)
```

`RuleCodec` używa wbudowanego `org.json`. Przykład formatu:

```
[
  {"package": "com.example.camera", "single": true},
  {"package": "com.example.clock", "single": false}
]
```

To przykładowe identyfikatory, nie gwarantowane nazwy aplikacji na tablecie. Pakiet identyfikuje aplikację, a `single` określa sposób jej uruchomienia. W JSON nie ma przyznanych uprawnień, ikony ani nazwy widocznej dla człowieka; te dane pochodzą z PackageManager i systemu polityk.

Walidator zabrania pustej listy bez włączonej strony, duplikatów, ręcznego dodawania Stanowiska i wskazania pakietu spoza aktualnie dostępnej listy. `decode()` oczekuje prawidłowego JSON z wymaganymi polami; nie naprawia dowolnie uszkodzonego zapisu.

Źródło: AppPolicy.kt.

Schemat i migracje

Profesor pyta: jak aktualizacja zachowuje starą konfigurację?

Room otwiera bazę wersji 3. `WorkspaceApp` rejestruje migracje `MIGRATION_1_2` i `MIGRATION_2_3`. Pierwsza dodaje dwie kolumny niewygaszania, druga dwie kolumny pasków. Każda nowa kolumna ma SQL-owe `NOT NULL DEFAULT 0`.

```
wersja 1: polityka i audyt
    ↓ dodanie keepScreenOn i appliedKeepScreenOn
wersja 2
    ↓ dodanie hideSystemBars i appliedHideSystemBars
wersja 3
```

W `app/schemas/.../1.json`, `2.json`, `3.json` znajdują się eksporty schematu, które pozwalają porównać strukturę. Nie ma `fallbackToDestructiveMigration()`, czyli świadomie skonfigurowanego kasowania bazy w zastępstwie brakującej migracji. Ogólny mechanizm i znaczenie testów migracji opisuje dokumentacja Room.

Profesor pyta: dlaczego hideSystemBars ma w Kotlinie true, a w SQL default 0?

To dwa różne momenty ustalania wartości. Nowy `PolicyRecord()` utworzony przez kod ma `hideSystemBars = true`. Rekord istniejący przed migracją 2→3 otrzymuje dodaną kolumnę z wartością `0`, czyli false. `@ColumnInfo(defaultValue = "0")` opisuje domyślną wartość schematu, a argument konstruktora opisuje tworzenie obiektu w Kotlinie. Zrównanie ich w opisie ukryłoby rzeczywistą różnicę zachowania nowych i aktualizowanych instalacji.

SharedPreferences

Profesor pyta: co nie trafiło do Room?

Nazwa preferences	Klucze	Przeznaczenie
`auth`	`credential`, `failures`, `blockedUntil`	Szyfrogram poświadczenia i ochrona przed kolejnymi próbami PIN-u
`runtime`	`session`, `keepScreenOn`, `hideSystemBars`, `hiddenLaunchers`	Odtwarzanie bieżącej sesji, zastosowane flagi i poprzedni stan obcych pulpitów
`web-page`	`enabled`, `title`, `url`, `additionalUrls`, `javaScript`, `singleSession`	Konfiguracja wbudowanej strony
`overlay-position`	`x`, `y`	Ułamkowa pozycja pływającego panelu

Większość istotnych zapisów używa `commit()` i sprawdza jego wynik. Pozycja nakładki używa `apply()`. W `apply()` silnika część zapisów podczas wycofywania jest wykonywana bez sprawdzenia zwróconej wartości; nie

należy więc opisywać wszystkich zapisów jako jednakowo kontrolowanych.

Źródła: PinStore.kt, WebPageConfig.kt, ExitOverlayService.kt.

Profesor pyta: czy strona też ma rewizję zapisaną i zastosowaną?

Nie. To ważny wyjątek od modelu polityki aplikacji. `saveWeb()` zapisuje konfigurację bezpośrednio w `web-page`, a `WebPageStore` publikuje nowy `StateFlow`. Przełącznik włączenia w formularzu zapisuje od razu. Pozostałe pola zapisuje przycisk. Otwarte `WebActivity` pobiera kopię konfiguracji w `onCreate`; pełna nowa konfiguracja obowiązuje przy następnym utworzeniu strony. Zmiany strony nie podnoszą `PolicyRecord.revision`.

Dziennik i reaktywne odczyty

Profesor pyta: ile zdarzeń przechowujemy i co widzimy?

`AuditEvent` ma automatyczny `id`, czas w milisekundach od epoki, `kind` i `detail`. `audit()` wykonuje insert, a następnie trim. Baza jest przycinana do 1000 najnowszych identyfikatorów, natomiast strumień widoczny dla UI zwraca 80 ostatnich rekordów. To trzy różne pojęcia: zapis, retencja i zakres prezentacji.

`UiText` tłumaczy kod zdarzenia na polską etykietę. Tylko wybrane detale pokazuje użytkownikowi, np. pakiet i tryb uruchomienia. Szczegóły wyjątków bywają przechowywane jako nazwy klas, a nie stosy wywołań. Dziennik jest lokalny i nie jest kryptograficznie chronionym, zewnętrznym rejestrem audytowym.

Profesor pyta: jak zmiana w bazie dociera do ekranu?

```
DAO.observe() / DAO.events()
       ↓ Flow
ViewModel.stateIn(viewModelScope, WhileSubscribed(5000), wartość początkowa)
       ↓ StateFlow
collectAsStateWithLifecycle()
       ↓ stan Compose
ponowne obliczenie odpowiednich części interfejsu
```

Polityka startuje od `null`, logi od pustej listy. `WhileSubscribed(5000)` oznacza opóźnienie zatrzymania zbierania upstream po utracie subskrybentów; nie odświeżanie co pięć sekund. Lista programów jest odświeżana osobno przez PackageManager. `refresh()` anuluje poprzednie zadanie, a brak możliwości odczytu uprawnień konkretnego pakietu zapisuje jako `null`, żeby odróżnić błąd od liczby zero.

Współbieżność i spójność

Profesor pyta: czy Mutex oznacza transakcję całego urządzenia?

Nie. Silnik ma własny `Mutex`, `PinStore` własny, a UI dodatkowo flagę `busy`. Mutex porządkuje operacje wykonywane przez dany obiekt w procesie. Nie łączy wywołań Androida, preferences i Room w jedną atomową

transakcję i nie przetrwa restartu.

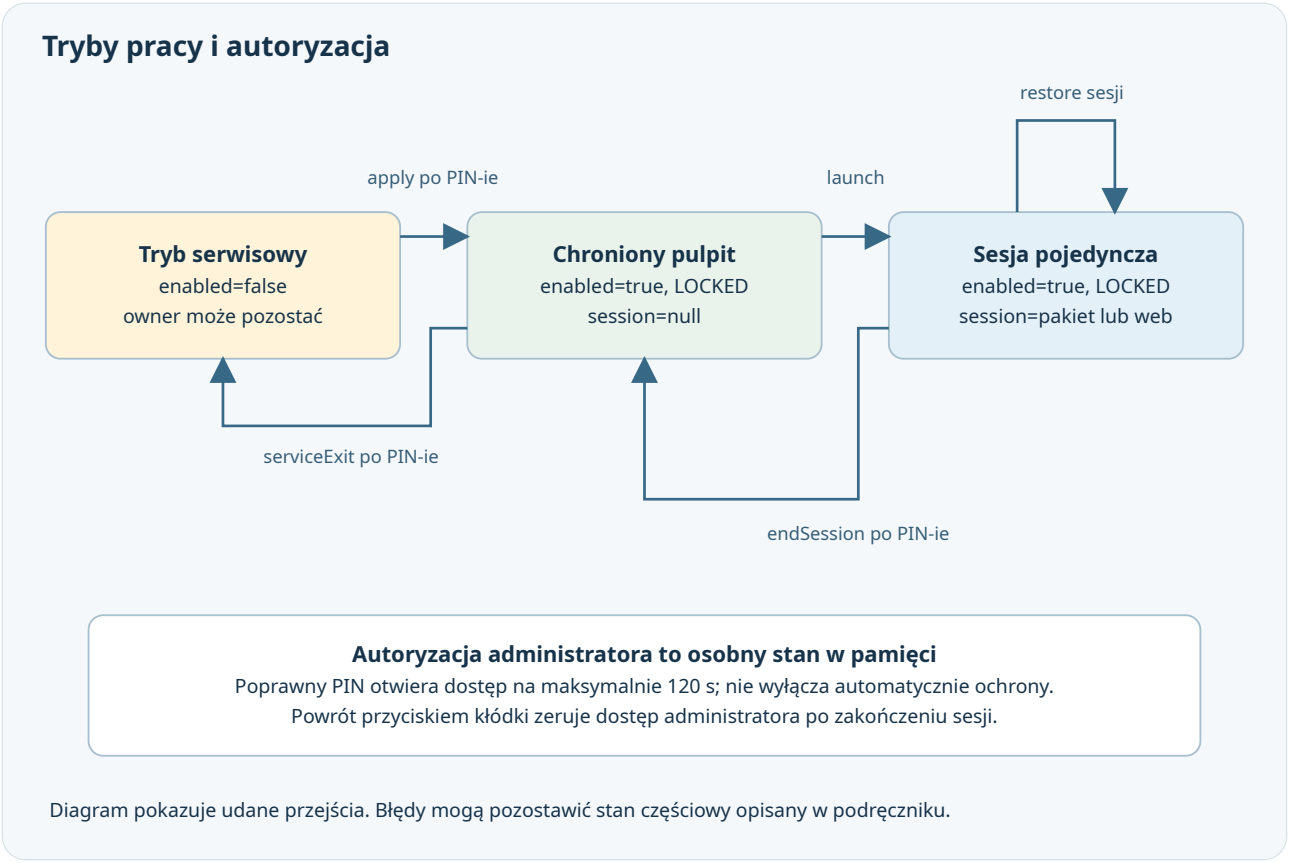
`AppPermissionManager.configure()` nie używa mutexu silnika, choć sprawdza administratora, właściciela i brak sesji. Stan Androida może też zmienić się niezależnie od procesu. Stąd odczyt kontrolny po zmianie, kopie zastosowanej polityki i logika odtwarzania.

Profesor pyta: jaki jest najtrudniejszy przypadek awarii?

Utrata procesu pomiędzy zapisem stanu lokalnego a dokończeniem zmian w Androidzie. Przykładowo sesja jest zapisywana przed zmianą polityki uruchamiania, a serwis utrwala `enabled=false` przed zniesieniem ograniczeń. To świadome uporządkowanie kroków pod określony cel, lecz nie usuwa wszystkich stanów częściowych. Ich konsekwencje i proponowane ulepszenia analizuje rozdział 8.

4. Egzekwowanie polityki i przebiegi operacji

Warunki chronionej pracy



Tryby pracy i odrębny stan autoryzacji administratora

Profesor pyta: jakie trzy stany najłatwiej pomylić?

Rolę właściciela urządzenia, intencję utrzymywania ochrony i rzeczywisty stan Lock Task. W kodzie odpowiadają im `engine.owner`, `PolicyRecord.enabled` i `engine.lockState`. Kafelek pulpitu pokazuje pełną ochronę dopiero dla połączenia: właściciel, aktywna polityka i `LOCK_TASK_MODE_LOCKED`.

Warunek	Co mówi	Czego sam nie zapewnia
`owner == true`	System uznaje Stanowisko za Device Owner	Że zastosowano konfigurację
`enabled == true`	Aplikacja ma utrzymywać i odtwarzać politykę	Że wszystkie wywołania Androida właśnie się powiodły
`lockState == LOCKED`	System działa w blokowanym trybie zadań	Że zapisany szkic jest już zastosowany
`session != null`	Trwa sesja jednej aplikacji lub strony	Że użytkownik ma dostęp administratora
`authorized() == true`	Nie minął termin autoryzacji w procesie	Że zniesiono ograniczenia urządzenia

Wszystkie opisane niżej metody znajdują się w PolicyEngine.kt, o ile nie wskazano innego pliku.

Zapis i zastosowanie

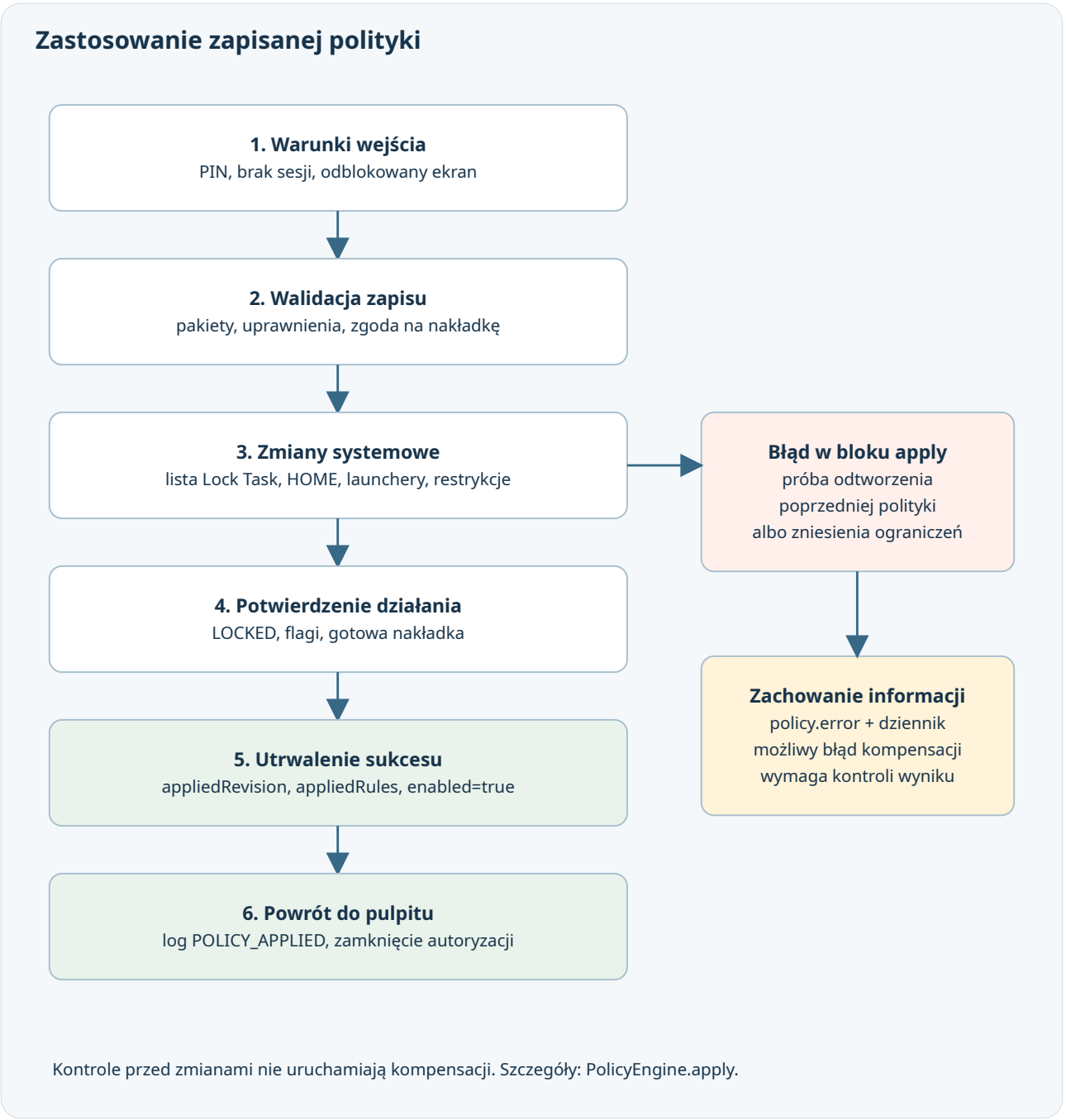
Profesor pyta: co robi save?

Pod blokadą mutexu sprawdza autoryzację, waliduje reguły względem aktualnej listy programów i dostępności strony, pobiera poprzedni rekord albo tworzy domyślny. Zapisuje numer rewizji powiększony o jeden, nowe reguły i dwie opcje ekranu. Czyści `error` i dopisuje `POLICY_SAVED`. Nie ustawia `enabled=true` ani nie wywołuje `startLockTask()`.

Profesor pyta: co dokładnie robi przycisk zastosowania?

`apply(activity)` wykonuje następujący przebieg:

1. Sprawdza administratora, skonfigurowany PIN, brak aktywnej sesji pojedynczej i odblokowany ekran Androida.
2. Odczytuje zapisaną politykę, dekoduje JSON i ponownie sprawdza dostępność aplikacji.
3. Wymaga przygotowanych uprawnień wszystkich wybranych pakietów i zezwolenia na nakładkę.
4. `setPolicy()` ustawia reguły w Androidzie, HOME, ukrycie innych pulpitów i trzy ograniczenia użytkownika.
5. `startProtection()` wywołuje `startLockTask()` na aktywności i czeka na `LOCK_TASK_MODE_LOCKED`.
6. Utrwala zastosowane flagi ekranu w runtime i ustawia własne okno.
7. Uruchamia nakładkę i czeka na jej potwierdzenie gotowości.
8. Ponownie sprawdza stan Lock Task i dopiero wtedy zapisuje zastosowaną kopię polityki z `enabled=true`.
9. Dodaje `POLICY_APPLIED` z numerem rewizji. Kod UI zamyka bieżącą autoryzację administratora.



Przebieg zastosowania zapisanej konfiguracji

Profesor pyta: dlaczego walidujemy drugi raz, skoro save już sprawdziło listę?

Między zapisem a zastosowaniem aplikację można usunąć, zaktualizować lub zmienić konfigurację WWW. Walidacja przy zapisie zapewnia poprawność szkicu w tamtym momencie. Walidacja przy zastosowaniu dotyczy stanu bieżącego. Jeszcze inną rolę ma kontrola wyniku systemowego po zmianie.

Operacje systemowe

Profesor pyta: co naprawdę ustawia setPolicy?

Lista Lock Task zawsze zawiera pakiet Stanowiska. Dodatkowe pakiety pochodzą z przekazanych reguł. Silnik ustawia persistent preferred HOME na `MainActivity`, ukrywa inne launchery i dodaje `DISALLOW_SAFE_BOOT`,

`DISALLOW_ADD_USER` oraz `DISALLOW_CONFIG_DATE_TIME`.

Opcje Lock Task są składane jako maska bitowa:

`single`	`hideBars`	**Włączane cechy**
false	false	BLOCK_ACTIVITY_START_IN_TASK, HOME, SYSTEM_INFO
false	true	BLOCK_ACTIVITY_START_IN_TASK
true	false	BLOCK_ACTIVITY_START_IN_TASK, SYSTEM_INFO
true	true	BLOCK_ACTIVITY_START_IN_TASK

Zablokowanie uruchamiania niedozwolonych aktywności w zadaniu jest więc włączane zawsze. Obsługa HOME jest dodawana tylko dla trybu wielu aplikacji przy widocznych paskach. Kod nie dodaje cech OVERVIEW ani NOTIFICATIONS. Następnie odczytuje listę pakietów, maskę i aktualny HOME, porównując je z oczekiwaniem. Nie odczytuje w tym samym warunku weryfikacyjnym każdej dodanej restrykcji użytkownika.

Profesor pyta: dlaczego zachowujemy poprzedni stan obcych pulpitów?

`secureOtherLaunchers()` wyszukuje HOME, pomija własny pakiet i kandydatów z ujemnym priorytetem. W `runtime.hiddenLaunchers` zapisuje mapę pakiet→poprzednia widoczność, zanim rozpocznie ukrywanie. Dzięki temu wyjście serwisowe może odtworzyć poprzedni stan, także jeśli aplikacja została w międzyczasie uruchomiona ponownie. Nie zakłada, że każdy launcher był początkowo widoczny.

Przywracanie poprzedniego stanu ukrycia jest jawnie modelowane. Dla trzech restrykcji użytkownika analogicznej kopii stanu początkowego nie ma: serwis je usuwa. Ten szczegół ma znaczenie przy rozszerzaniu zestawu polityk.

Profesor pyta: po co pamiętamy korzeń zadania?

`startProtection()` zapisuje `taskId` i słabą referencję do Activity uruchamiającej Lock Task. `attach()` może podpiąć nową instancję Activity tego samego zadania. Słaba referencja nie utrzymuje starego ekranu przy życiu. To potrzebne przy wychodzeniu, ponieważ `stopLockTask()` ma znaczenie w kontekście aktywności, która rozpoczęła tryb. Zachowanie platformy opisuje dokumentacja zatrzymywania Lock Task.

Silnik czeka na zmianę stanu maksymalnie 5 sekund, odczytując stan co 50 ms. To asynchroniczne oczekiwanie korutyny, a nie blokowanie wątku przez `Thread.sleep()`.

Błąd zastosowania

Profesor pyta: co dzieje się, jeśli Android odrzuci część zmian?

W bloku obsługi błędu `apply()` podejmuje próbę wycofania. Jeśli poprzednia polityka była aktywna, odtwarza `appliedRules`, flagi i nakładkę. Jeśli nie była aktywna, wyłącza własne flagi i usługę, próbuje zakończyć Lock

Task, czyści listę, persistent HOME, przywraca launchery i usuwa trzy restrykcje.

Błąd samego wycofania dopisuje jako suppressed exception i rejestruje `POLICY_ROLLBACK_ERROR`. Następnie zachowuje pierwotny rekord polityki z opisem błędu, loguje `POLICY_ERROR` i przekazuje wyjątek wyżej. UI prezentuje komunikat przez ViewModel.

To **próba kompensacji skutków**, nie transakcja systemowa z gwarancją pełnego cofnięcia. Warunki sprawdzane przed blokiem `try`, np. brak uprawnień czy nakładki, nie uruchamiają tej ścieżki wycofywania, bo zmiany systemowe nie zostały jeszcze rozpoczęte przez tę operację.

Uruchomienie aplikacji i sesja

Profesor pyta: co dzieje się po naciśnięciu kafelka programu?

`launch()` wymaga braku innej sesji i znajduje dokładnie jedną regułę dla pakietu. Przy aktywnej polityce korzysta z `appliedRules`, w pozostałym przypadku z zapisanych `rules`. Pobiera systemowy Intent uruchomienia.

Jeżeli ochrona jest wyłączona albo nie ma Device Owner, zwykłą regułę wieloaplikacyjną można uruchomić bez pełnej ochrony. Reguła pojedynczej sesji zostanie wtedy odrzucona. To użyteczna możliwość demonstracji pulpitu, ale jej stan jest jawnie oznaczony jako brak pełnej ochrony.

Przy ochronie silnik wymaga przygotowanych uprawnień. Dla sesji pojedynczej najpierw uruchamia nakładkę w trybie kłódki, zapisuje nazwę pakietu w `runtime.session`, zawęża politykę i uruchamia program z `NEW_TASK | CLEAR_TASK`. `CLEAR_TASK` oznacza możliwość usunięcia dotychczasowego stosu aktywności zadania; nie oznacza kasowania danych aplikacji z dysku. Zwykłe uruchomienie także w tej chronionej ścieżce otrzymuje te flagi.

Profesor pyta: jak wygląda sesja strony?

`launchWeb()` wymaga włączonej, poprawnej strony oraz aktywnej ochrony z Device Owner. W pojedynczej sesji zapisuje specjalny znacznik `stanowisko:web` i ustawia listę tylko z własnym pakietem. W wielu aplikacjach odtwarza listę zastosowanych reguł. Uruchamia `WebActivity` w osobnym zadaniu. Nie dodaje pakietu przeglądarki do listy: WebView jest widokiem wewnątrz Stanowiska.

Powrót i odtwarzanie

Profesor pyta: jak kończy się sesję jednej aplikacji?

Nakładka otwiera `AuthActivity`. Po poprawnym PIN-ie aktywna sesja jest kończona przez `endSession()`: silnik tymczasowo pozostawia na liście Lock Task tylko własny pakiet, przywraca reguły wielu aplikacji, usuwa `runtime.session` i aktualizuje nakładkę. Dla strony dodatkowo usuwa jej zadanie z `appTasks`.

PIN pozwalający wyłącznie wrócić do pulpitu (`EXTRA_RETURN_DESKTOP`) nie zostawia otwartego panelu

administratora: autoryzacja jest zerowana przed przejściem. Zwykłe wejście przez przycisk administratora może otworzyć panel. Szczegóły zawiera AuthActivity.kt.

Profesor pyta: czy po restarcie trzeba zapisać konfigurację od nowa?

Nie. `BootReceiver` dla uruchomienia systemu lub wymiany własnego pakietu sprawdza Device Owner i `enabled`, po czym prosi o otwarcie pulpitu. Główne odtwarzanie wykonuje `MainActivity.onResume()` przez `restore()`.

`restore()` korzysta z **zastosowanych**, a nie dopiero zapisanych reguł. Dla sesji natywnej wybiera jej jeden pakiet, dla sesji WWW pustą listę dodatkowych pakietów, a przy braku sesji pełne `appliedRules`. Odtwarza politykę, Lock Task, flagi i nakładkę; następnie próbuje otworzyć cel sesji. Przy zablokowanym keyguardzie odkłada działania do późniejszego wywołania. Błąd zachowuje z prefiksem `Odtwarzanie:` i loguje `RESTORE_ERROR`.

To mechanizm ponawiany przy powrocie pulpitu, nie stały watchdog i nie obietnica natychmiastowego przywrócenia w każdym wariancie oprogramowania producenta. BootReceiver.kt używa `goAsync()` i kończy pracę `pending.finish()`.

Wyjście serwisowe

Profesor pyta: czym wyjście serwisowe różni się od zamknięcia panelu?

Zamknięcie panelu zeruje autoryzację. `serviceExit()` zmienia politykę urządzenia: wymaga administratora i braku sesji, zapisuje `enabled=false`, wyłącza flagi oraz nakładkę, próbuje zakończyć Lock Task i przywraca środowisko systemowe.

Jeśli korzeń zadania jest niedostępny lub `stopLockTask()` nie zadziała, po próbie oczekiwania do 1,5 sekundy silnik odbiera pakietom allowlistę, a następnie sprawdza stan. Czyści persistent HOME, odtwarza zapisane ukrycie pulpitów i usuwa trzy restrykcje. **Nie usuwa Device Owner i nie przywraca wcześniejszych decyzji o uprawnieniach aplikacji.** Nie czyści zapisanej konfiguracji ani PIN-u.

Wcześniejszy zapis `enabled=false` zapobiega automatycznemu ponownemu nakładaniu ochrony po restarcie. Jednocześnie awaria w środku tego procesu może pozostawić część systemowych ograniczeń przy wyłączonym zamiarze odtwarzania; dlatego stan serwisu trzeba potwierdzić na urządzeniu.

5. PIN administratora i uprawnienia aplikacji

Inicjalizacja i autoryzacja

Profesor pyta: jaki PIN ma świeża instalacja?

`1111`. `WorkspaceApp` uruchamia leniwe zadanie inicjalizacji, a `PinStore.initializeDefault()` zapisuje tę wartość tylko wtedy, gdy preferences nie zawierają `credential`. `MainActivity` czeka na ukończenie inicjalizacji przed pokazaniem właściwego interfejsu. Aktualizacja zachowująca dane nie nadpisuje istniejącego poświadczenia.

W ekranie pozostała ścieżka ręcznego tworzenia PIN-u przy `configured=false`, ale normalny udany start po inicjalizacji ma już domyślny PIN. Błąd inicjalizacji prowadzi do komunikatu błędu uruchomienia. Nie należy więc instruować, że każda świeża instalacja zaczyna od formularza tworzenia PIN-u.

Źródła: WorkspaceApp.kt, MainActivity.kt, PinStore.kt.

Profesor pyta: czy 1111 jest kodem awaryjnym?

Nie. Jest początkowym poświadczeniem, które po zmianie nie otrzymuje dodatkowej ścieżki weryfikacji. `verify()` sprawdza zapisany materiał kryptograficzny. Kod nie ma porównania „właściwy PIN albo 1111". Zmiana PIN-u powinna być elementem przygotowania urządzenia, bo znany czterocyfrowy kod początkowy daje dostęp każdemu, kto go zna.

Profesor pyta: czym jest stan administratora?

Po poprawnym PIN-ie `AuthActivity` ustawia `adminUntil = elapsedRealtime() + 120_000`. Jest to termin ważności w pamięci procesu, liczony monotonicznie od uruchomienia systemu. `authorized()` porównuje go z bieżącym `elapsedRealtime()`. To maksymalnie dwie minuty od autoryzacji, a nie automatycznie odnawiana dwuminutowa bezczynność.

Globalne callbacki Activity mogą wyzerować termin wcześniej, kiedy zatrzymuje się aktualnie śledzona aktywność i nie chodzi o zmianę konfiguracji. Kod dba, aby zatrzymanie starego pulpitu nie odebrało dostępu nowemu oknu. Ochrona uprawnień nie opiera się wyłącznie na widoczności przycisków: wrażliwe operacje wywołują `requireAdmin()`.

Profesor pyta: co chroni ekran wpisywania kodu?

`AuthActivity` ustawia `FLAG_SECURE` oraz `setHideOverlayWindows(true)`. Ekran administracyjny w `WorkspaceScreen` ustawia `FLAG_SECURE` przy aktywnym administratorze lub braku PIN-u. Pole maskuje cyfry, wymusza numeryczny typ klawiatury i przyjmuje najwyżej 12 znaków z zakresu ASCII `0`–`9`. Minimum 4 jest sprawdzane przy zapisie, nie na każdym wpisanym znaku.

Te ustawienia ograniczają przechwytywanie w zwykłych mechanizmach systemowych; nie zastępują testu

fizycznego urządzenia ani nie stanowią ochrony przed przejętym systemem. Nie ma wymogu biometrii ani ponownego uwierzytelnienia klucza Keystore przy każdej operacji.

Przechowywanie PIN-u

Profesor pyta: czy PIN jest zapisany jako zwykły tekst?

Nie. Najpierw powstaje pochodna PBKDF2, a następnie materiał do weryfikacji jest szyfrowany kluczem z Android Keystore. Kolejność ma znaczenie:

```
PIN i losowa sól 32 bajty
        ↓ PBKDF2WithHmacSHA256, 600 000 iteracji, wynik 256 bitów
32 bajty soli || 32 bajty pochodnej
        ↓ AES/GCM/NoPadding, klucz Android Keystore workspace.pin
IV || szyfrogram z tagiem uwierzytelniającym
        ↓ Base64
auth.credential w prywatnych SharedPreferences
```

`key()` pobiera istniejący klucz AES albo tworzy go przez dostawcę `AndroidKeyStore`. Kod nie wymusza StrongBox i nie sprawdza poziomu sprzętowego zabezpieczenia. Keystore zarządza kluczem bez jego zwykłego eksportu do pliku aplikacji; dostępność ochrony sprzętowej zależy od urządzenia. Źródło platformowe: Android Keystore.

Profesor pyta: dlaczego sól, PBKDF2 i AES-GCM jednocześnie?

To różne zadania. Sól sprawia, że identyczny PIN nie daje identycznej pochodnej dla każdej instalacji. PBKDF2 zwiększa koszt obliczenia kandydata. AES-GCM chroni poufność i integralność zapisanych soli oraz pochodnej pod kluczem Keystore. Base64 wyłącznie koduje bajty jako tekst; nie jest szyfrowaniem.

Jest to uzasadnienie wynikające z właściwości użytych mechanizmów. Samo „600 tysięcy iteracji” nie dodaje entropii do czterech cyfr. PIN czterocyfrowy ma tylko 10 000 możliwych wartości, dlatego znaczenie mają ograniczanie prób i zmiana kodu początkowego.

Profesor pyta: co robi verify krok po kroku?

Pod własnym mutexem, na `Dispatchers.IO`, sprawdza czas blokady i obecność poświadczenia. Dekoduje Base64, traktuje pierwsze 12 bajtów jako IV GCM, odszyfrowuje pozostałą część, wydziela 32-bajtową sól i ponownie wylicza PBKDF2 dla kandydata. Porównuje pochodne przez `MessageDigest.isEqual`. Następnie zeruje część buforów i zatwierdza liczniki prób.

`PinCrypto` czyści `PBEKeySpec` oraz otrzymaną tablicę znaków. Nie oznacza to wyczyszczenia każdej kopii PIN-u w zarządzanej pamięci: interfejs i metody operują również na niemodyfikowalnych `String`. Utrata lub unieważnienie klucza Keystore nie uruchamia zaprojektowanego tu resetu poświadczenia; błąd odszyfrowania jest błędem operacji.

Źródła: PinCrypto.kt, PinStore.kt.

Ograniczanie prób

Profesor pyta: kiedy pojawia się blokada i ile trwa?

Liczba kolejnych niepowodzeń	Opóźnienie po tej próbie
1–4	0 s
5	30 s
6	60 s
7	120 s
8	240 s
9	480 s
10 i więcej	900 s, czyli 15 min

Wzór to `30_000 × 2^min(failures−5, 5)` ograniczony do `900_000` ms. Poprawna weryfikacja zeruje licznik i termin blokady. Próba podczas blokady kończy się wyjątkiem z czasem oczekiwania, zanim zostanie obliczona pochodna.

Termin blokady jest utrwalany jako `System.currentTimeMillis()`, więc przetrwa restart procesu. Ten zegar zależy jednak od czasu kalendarzowego. Dwie minuty dostępu administratora korzystają z innego zegara, monotonicznego, i nie przetrwają utraty procesu. Zmiana zegara kalendarzowego jest jednym z powodów, dla których ograniczenia bezpieczeństwa trzeba oceniać razem z konfiguracją systemu.

Przygotowanie uprawnień programów

Profesor pyta: co oznacza „uprawnienia przygotowane”?

`AppPermissionManager.list()` czyta uprawnienia zadeklarowane w pakiecie i wybiera tylko te z bazowym poziomem `PROTECTION_DANGEROUS`. Nie ma tu wszystkich ustawień Androida, dostępu Accessibility, uprawnienia nakładki, wyjątków optymalizacji baterii ani dowolnych ustawień producenta.

Model ma `name`, polską etykietę systemową, efektywny stan `granted` i stan polityki DPM. Właściwość `prepared` oznacza:

```
granted || policyState == PERMISSION_GRANT_STATE_DENIED
```

A zatem przygotowanie oznacza przyznane uprawnienie **albo jawną odmowę administracyjną**. Nie oznacza przyznania wszystkiego. Zwykłe nieprzyznane uprawnienie w stanie DEFAULT pozostaje nieprzygotowane, bo

aplikacja mogłaby próbować otworzyć dialog systemowy podczas chronionej pracy.

Źródło: AppPermissions.kt.

Profesor pyta: dlaczego decyzje zapadają przed uruchomieniem ochrony?

Program, który po pierwszym otwarciu żąda dostępu lub pokazuje kreator, może wymagać dodatkowych ekranów systemowych. Projekt rozdziela przygotowanie od właściwej sesji. `apply()` sprawdza przygotowanie całej listy; `launch()` sprawdza konkretny cel. Jeśli po aktualizacji w czasie ochrony pojawią się nowe nieprzygotowane uprawnienia, pulpit może skierować administratora do autoryzacji i dialogu ustawień.

`prepareApp()` uruchamia program poza ochroną, ale wymaga administratora, braku sesji, `LOCK_TASK_MODE_NONE` i wyłączonej polityki. Pozwala przejść pierwsze uruchomienie i ustawienia specjalne. Nie jest obejściem ochrony dostępne zwykłemu użytkownikowi.

Kolejność i kontrola zmian

Profesor pyta: po co osobna PermissionGrantPlan?

Lokalizacja ma zależności. Plan odrzuca zgodę na dokładną lokalizację przy jednoczesnej odmowie przybliżonej, jeśli ta występuje w mapie, oraz zgodę na lokalizację w tle bez zgody na lokalizację podczas działania. Walidacja otrzymuje aktualne efektywne zgody połączone ze zmianami, więc bierze pod uwagę oba źródła.

Zmiany odmów są wykonywane przed zgodami. W obrębie zgód kolejność to przybliżona lokalizacja, dokładna, pozostałe, a następnie lokalizacja w tle. Przy odmowach ranga jest odwrócona, aby najpierw odebrać uprawnienie zależne. Nazwa uprawnienia stabilizuje pozostałą kolejność.

Profesor pyta: czy Android musi wykonać każdą decyzję?

Nie. `configure()` wymaga administratora, Device Owner, braku aktywnej sesji oraz kluczy należących do zadeklarowanych uprawnień. Sprawdza wynik `setPermissionGrantState`, a potem odczytuje ponownie zarówno stan polityki, jak i efektywną zgodę. Platforma ogranicza możliwość zmiany niektórych uprawnień w zależności od ich rodzaju, roli zarządzania i konfiguracji. Nie należy traktować samego poziomu dangerous jako gwarancji swobodnego nadania. Źródło: DevicePolicyManager).

Przy niepowodzeniu menedżer próbuje odwrócić już wykonane zmiany. Odtwarza najpierw poprzednią efektywną zgodę, a następnie poprzedni stan polityki. Ta ścieżka także może zawieść; nie ma wokół każdej czynności wycofania osobnego przechwycenia błędu. Powodzenie zapisuje jako `APP_PERMISSIONS_CHANGED`, a obsłużony błąd jako `APP_PERMISSION_ERROR`.

Profesor pyta: czy uprawnienia wracają do dawnego stanu po wyjściu serwisowym?

Nie. Zakończenie sesji i serwis zarządzają ograniczeniami pracy urządzenia, a nie przywracaniem wszystkich zgód aplikacji. Długotrwałe decyzje GRANTED i DENIED pozostają polityką, dopóki administrator lub inny

uprawniony mechanizm ich nie zmieni. To trzeba uwzględnić, planując przekazanie urządzenia do innego zastosowania.

6. Interfejs użytkownika, nakładka i strona internetowa

Compose i stan ekranu

Profesor pyta: co decyduje, czy widzę pulpit, czy panel?

`WorkspaceScreen` obserwuje politykę, listę aplikacji, uprawnienia, WWW, dziennik, komunikat i `busy`. Oprócz tego odczytuje bieżącą autoryzację i sesję. Jeśli nie ma sesji, PIN istnieje, a administrator nie jest autoryzowany, wyświetla `WorkspaceLauncher` i kończy budowanie reszty ekranu. W innych gałęziach pokazuje stan sesji, konfigurację początkową albo panel administratora.

Raz na sekundę, tylko przy stanie lifecycle RESUMED, zwiększa `tick`. Ponieważ `adminUntil` i runtime nie są wszędzie strumieniami, to wymusza ponowny odczyt autoryzacji i sesji. `generation` jest zwiększane także przy `onResume` i odtworzeniu. Stan administratora na ekranie nie jest zatem jedynie zatrzaśnięty w chwili otwarcia.

Źródła: WorkspaceScreen.kt, MainActivity.kt.

Profesor pyta: co dzieje się z niezapisanym formularzem?

Reguły, niewygaszanie i paski są lokalnym szkicem `remember(policy?.revision)`. Zmiana rewizji odtwarza szkic z zapisu. Formularz WWW używa `remember(savedPage)`. Nie są to pola `rememberSaveable` ani trwały zapis przy każdym dotknięciu. Rekreacja kompozycji i aktywności może spowodować utratę niezapisanych zmian. Informację o wykonanym zapisie daje dopiero ukończona operacja i jej komunikat.

Profesor pyta: dlaczego komunikaty czasem znikają, a czasem zostają?

`NoticeKind.INFORMATION` znika po czterech sekundach. `CONFIGURATION` i `ERROR` pozostają do zastąpienia albo wyczyszczenia przez kolejną operację. Wygaszanie używa `compareAndSet`, więc stare zadanie nie usuwa nowego komunikatu. Błędy mają semantykę `LiveRegionMode.Assertive`, pozostałe `Polite`, aby usługa dostępności mogła je ogłaszać.

Źródła: WorkspaceViewModel.kt, WorkspaceNotice.kt, VisualDesign.kt.

Układ, ikony i dane systemowe

Profesor pyta: jak interfejs dostosowuje się do telefonu i tabletu?

Launcher mierzy dostępną przestrzeń przez `BoxWithConstraints`. Szerokość co najmniej 600 dp oznacza układ szeroki, a wysokość poniżej 500 dp układ kompaktowy. W wąskim widoku są dwie kolumny; w szerokim ich liczba wynika z szerokości i mieści się w zakresie 2–4. Etykieta administratora jest dodawana od 420 dp. Rozmiary kafelków, ikon i odstępów zmieniają się razem z układem.

Panel administratora ma listę leniwą o maksymalnej szerokości 840 dp, uwzględnia bezpieczne obszary i klawiaturę. Motyw Compose jest ciemny, z turkusowym akcentem. Bazowy motyw XML ustawia także wygląd okna, zanim Compose narysuje treść.

Profesor pyta: czy na pulpicie widzimy prawdziwe ikony aplikacji?

W panelu listy aplikacji tak: ikonę pobiera PackageManager i konwertuje do bitmapy. Kafelki launchera używają własnych rysunków `WorkspaceGlyph`, dobranych przez fragmenty nazwy pakietu, np. `camera`, `clock`, `contacts`. Dla pozostałych używają ogólnej siatki. Nie ma tu automatycznej klasyfikacji znaczenia każdej zainstalowanej aplikacji.

Zegar odczytuje lokalny czas i strefę urządzenia, formatuje po polsku datę i sekundy. Bateria pochodzi z `ACTION_BATTERY_CHANGED`, w tym ze sticky intent od razu po rejestracji. Odbiornik jest rejestrowany w stanie STARTED i usuwany w `finally` po zakończeniu zbierania. Nie jest potrzebny zewnętrzny serwis zegara ani osobne uprawnienie do procentu baterii.

Pływające menu

Profesor pyta: dlaczego menu jest usługą, a nie częścią pulpitu?

Musi być widoczne również nad inną aplikacją. `ExitOverlayService` tworzy przez WindowManager okno `TYPE_APPLICATION_OVERLAY`, a nie zwykły element Compose wewnątrz `MainActivity`. Wymaga systemowego zezwolenia `SYSTEM_ALERT_WINDOW`. Usługa foreground ma kanał powiadomienia i manifestowy typ `specialUse` z opisem celu.

Przyciski są zwykłymi Android Views. Nakładka ma `FLAG_NOT_FOCUSABLE`, więc nie przejmuje zwykłego fokusu klawiatury. Przycisk domu w chwili kliknięcia ponownie sprawdza `engine.session`: otwiera pulpit albo PIN z żądaniem zakończenia sesji. Zachowanie nie polega na zaufaniu do aktualnie narysowanej ikony.

Źródło: ExitOverlayService.kt.

Profesor pyta: skąd silnik wie, że menu rzeczywiście powstało?

Przed wywołaniem usługi tworzy nowe `CompletableDeferred<Unit>` w `ExitOverlayService.ready`. Usługa kończy je sukcesem po dodaniu lub aktualizacji okna, a wyjątkiem przy błędzie. Silnik czeka maksymalnie pięć sekund. Samo wywołanie `startForegroundService` nie jest więc uznane za wystarczające potwierdzenie.

Usługa zwraca `START_STICKY`, co pozwala systemowi próbować odtworzyć ją po zakończeniu. Nie jest to gwarancja nieprzerwanej obecności okna. Kod przewiduje także ponowne wywołania usługi przez silnik.

Profesor pyta: jak zachowuje pozycję po obrocie?

Współrzędne zapisuje jako ułamki 0–1 dostępnego zakresu ruchu. `FloatingPanelBounds` przelicza je na aktualne piksele, uwzględniając rozmiar panelu, paski, wycięcia i margines. `clamp()` utrzymuje panel w

granicach; wartości niefinitywne są zastępowane bezpiecznym ułamkiem. Uchwyt rozpoznaje przeciągnięcie dopiero po przekroczeniu systemowego progu ruchu.

Domyślna pozycja to prawa strona (`x=1`) i 15% zakresu pionowego. `onConfigurationChanged` przelicza szerokość i pozycję. Źródło obliczeń: FloatingPanelPosition.kt.

Profesor pyta: dlaczego nakładka znika nad panelem?

`WorkspaceApp` publikuje `workspaceUiVisible` . Własne Activity poza WebActivity traktuje jako okno pulpitu lub administracji. Usługa zbiera ten strumień i chowa panel, gdy nie jest potrzebny. Strona jest wyjątkiem, bo nadal musi mieć sterowanie. To koordynacja widoczności wewnątrz procesu, a nie wykrywanie wszystkich aktywności systemu.

Konfiguracja adresów WWW

Profesor pyta: co znaczy dozwolona domena w tej implementacji?

Precyzyjnie: dozwolony **origin**, czyli schemat, host i efektywny port. `WebAddress.origin()` usuwa ścieżkę, query i fragment, zmniejsza litery schematu i hosta oraz normalizuje domyślne porty 80 i 443. Odrzuca inne schematy niż HTTP/HTTPS, brak hosta, login w URL, port poza 1–65535, białe znaki na końcach i adres dłuższy niż 2048 znaków.

Skonfigurowano `https://example.org/panel`	Czy ma ten sam origin
`https://example.org/inne?x=1`	Tak
`https://EXAMPLE.org:443/a`	Tak
`http://example.org/panel`	Nie, inny schemat
`https://example.org:8443/panel`	Nie, inny port
`https://login.example.org/`	Nie, inny host
`https://example.org.evil.invalid/`	Nie, inny host
`mailto:user@example.org`	Nie, niedozwolony schemat

Nie ma porównania prefiksu tekstowego ani wildcardu subdomen. Każdy dodatkowy origin trzeba dodać osobnym adresem. Wpisanie jednej ścieżki nie ogranicza dostępu tylko do tej ścieżki.

Tytuł ma 1–60 znaków po normalizacji, a pole dodatkowych adresów najwyżej 16 384 znaki. Główny URL musi być poprawny przy włączonej stronie. Dodatkowe niepuste adresy są walidowane także przy stronie wyłączonej. Źródło: WebPageConfig.kt.

WebView i granice izolacji

Profesor pyta: co blokuje WebActivity?

Odrzuca nawigację poza dozwolone origins, nie otwiera linków w aplikacjach zewnętrznych, anuluje błędy certyfikatu TLS. Wyłącza dostęp do `file:` i `content:`, mieszane treści HTTPS/HTTP, wiele okien, automatyczne okna JavaScript, wybieranie i pobieranie plików. Odmawia żądań uprawnień strony oraz geolokalizacji. Nie udostępnia mostka `addJavascriptInterface`.

DOM storage jest włączony, a JavaScript domyślnie również. HTTP jest celowo dozwolone w manifeście dla stron, więc administrator może skonfigurować połączenie nieszyfrowane. Błąd certyfikatu HTTPS nie jest obchodzony przyciskiem „kontynuuj". Wszystkie te zachowania wynikają z WebActivity.kt.

Profesor pyta: dlaczego adres jest sprawdzany w kilku callbackach?

`shouldOverrideUrlLoading` kontroluje przejścia, `shouldInterceptRequest` obsługuje także główne żądania POST, a `onPageStarted` zatrzymuje rozpoczęte ładowanie niedozwolonej strony. Intercept zwraca wtedy lokalną odpowiedź HTTP 403. Aktualizacja komunikatu z callbacku żądania przechodzi przez `view.post`, bo to nie musi być wątek UI.

Platforma nie wywołuje `shouldOverrideUrlLoading` dla wszystkich nawigacji, w tym POST. Intercept przy przekierowaniu jest wywoływany dla początkowego URL, nie każdego kolejnego. Są to ograniczenia opisane w WebViewClient, które trzeba uwzględnić w testach przekierowań.

Profesor pyta: czy to zapora całego internetu?

Nie. W `shouldInterceptRequest` warunek odrzucenia dotyczy tylko `request.isForMainFrame`. Podzasoby, żądania skryptów, obrazy, połączenia aplikacji natywnych i cała polityka sieci urządzenia nie są objęte jednym globalnym filtrem tej listy. `onPageStarted.stopLoading()` też nie dowodzi, że przed zatrzymaniem nie nastąpiła żadna komunikacja. Z tego kodu wynika kontrola nawigacji WebView, nie gwarancja szczelnego filtra ruchu lub ochrony przed wyciekiem danych.

Profesor pyta: co stanie się po awarii strony?

Błąd głównego żądania pokazuje panel z ponowieniem i administratorem. Przy awarii procesu renderera `onRenderProcessGone` usuwa i niszczy WebView, zachowując Activity z komunikatem; ponowienie może wykonać `recreate()`. Wstecz przechodzi po historii; dopiero brak historii w trybie wielu aplikacji pozwala wrócić na pulpit. W pojedynczej sesji nie kończy jej bez PIN-u.

`onResume` ponownie sprawdza zgodność znacznika sesji, włączenie strony, właściciela i aktywny Lock Task. Jeśli warunki się nie zgadzają, kończy Activity. `onDestroy` usuwa i niszczy widok, ale nie ma tu procedury czyszczenia cookies, DOM storage i zalogowanej sesji WWW po zakończeniu pracy użytkownika.

Ograniczenia urządzeń producenta

Profesor pyta: co robi obsługa Lenovo?

`LenovoSystemMenu` rozpoznaje producenta Lenovo i dokładnie model oraz device `TB311FU`. Czyta ustawienie `enable_temp_zuifreeformbar`. W obecnym kodzie ekran wykorzystuje `supported` do pokazania informacji; `requiresPreparation` nie steruje blokadą zastosowania polityki. Klasa niczego nie zapisuje w ustawieniach producenta i nie wyłącza jego menu. To wykrycie znanego warunku wymagającego sprawdzenia na urządzeniu, a nie ogólna obsługa wszystkich tabletów Lenovo.

7. Budowanie, zależności i instalacja

Łańcuch narzędzi

Profesor pyta: co robi każde narzędzie i jakie wersje ma ten projekt?

Element	Wersja lub ustawienie	Zadanie
Gradle Wrapper	9.1.0	Uruchamia zadania budowania w przewidywalnej wersji
Android Gradle Plugin	9.0.1	Zna warianty Androida, manifest, zasoby, DEX i APK
Kotlin w AGP	Wbudowana obsługa; runtime rozwiązuje stdlib 2.2.10	Kompilacja kodu Kotlin i biblioteka standardowa
Compose compiler plugin	2.2.10	Obsługa kompilacji funkcji `@Composable`
KSP	2.3.4	Uruchamia procesor symboli generujący kod Room
`compileSdk`	36	API dostępne podczas kompilowania
`targetSdk`	36	Docelowy zestaw zachowań platformy
`minSdk`	34	Najstarszy dopuszczony system instalacji
Java source/target i Kotlin jvmTarget	17	Docelowy poziom kodu JVM przed konwersją do DEX
JDK użyty do kontroli	JetBrains Runtime 25.0.4	JVM uruchamiająca Gradle w tym środowisku

Źródła wersji: główny build.gradle.kts, moduł aplikacji, wrapper.

Profesor pyta: dlaczego nie ma pluginu org.jetbrains.kotlin.android?

AGP 9.0 włącza obsługę Kotlin bezpośrednio. Brak tego pluginu nie jest tu pominięciem języka. Nie należy automatycznie dodawać go według starszego szablonu, bo może kolidować z nowym mechanizmem. Źródło: migracja do Kotlin wbudowanego w AGP.

Profesor pyta: czy Java 17 i JDK 25 nie są sprzeczne?

Nie. Jedno to poziom kompilowanego kodu, drugie to JVM uruchamiająca narzędzia. Projekt ustawia target 17, a lokalne README zaleca JDK 25 dla Gradle. AGP 9.0 podaje JDK 17 jako minimum, Gradle 9.1.0 i Build Tools 36.0.0 jako zgodny zestaw. Gradle 9.1 dodał obsługę Java 25. W tej kontroli rzeczywiście użyto JBR 25.0.4. Źródła: zgodność AGP 9.0, Gradle 9.1.

Biblioteki aplikacji

Profesor pyta: za co odpowiada każda bezpośrednia zależność?

Deklaracja	Wersja	Rola w tym kodzie
`androidx.compose:compose-bom`	2025.12.00	Zestaw zgodnych ograniczeń wersji bibliotek Compose
`androidx.compose.material3:material3`	Z BOM; rozwiązane 1.4.0	Formularze, przyciski, dialogi, kolory i motyw
`androidx.activity:activity-compose`	1.12.2	`setContent`, integracja Activity i Compose, obsługa Wstecz
`lifecycle-viewmodel-compose`	2.10.0	Zależności ViewModel dla integracji z Compose
`lifecycle-runtime-compose`	2.10.0	Lifecycle w Compose, zbieranie stanu zgodne z cyklem życia
`room-runtime`	2.8.4	Baza, encje i DAO
`room-ktx`	2.8.4	API Room dla korutyn i strumieni
`room-compiler` przez `ksp`	2.8.4	Generowanie implementacji bazy i DAO podczas budowania
`kotlinx-coroutines-android`	1.10.2	Zadania, Flow, mutexy i Android Main dispatcher

BOM nie jest silnikiem wykonującym UI. Zarządza wersjami, a biblioteki faktycznie trafiają do grafu przez zależności. Przykładowo rozwiązany graf debug obejmuje Compose UI 1.10.0 i Kotlin stdlib 2.2.10. `room-compiler` jest procesorem budowania, a nie biblioteką potrzebną do generowania DAO na tablecie.

Repozytoria pluginów to Google, Maven Central i Gradle Plugin Portal; biblioteki modułów pochodzą z Google i Maven Central. `FAIL_ON_PROJECT_REPOS` zabrania dopisywania dowolnego repozytorium wewnątrz modułu. Nie ma katalogu wersji `libs.versions.toml`, dependency lockfile ani konfiguracji weryfikacji wszystkich artefaktów. Wrapper i instalatory mają własne sumy SHA-256, ale nie oznacza to kontroli sum każdej biblioteki Maven.

Przebieg budowania

Profesor pyta: co powstaje między plikiem kt a APK?

```
Gradle i pluginy
    ↓ konfiguracja wariantu debug
manifest + zasoby XML → scalanie i przetwarzanie zasobów
Kotlin + encje Room → KSP → wygenerowana implementacja bazy i DAO
    ↓ kompilacja Kotlin z pluginem Compose
klasy JVM + biblioteki
    ↓ D8 / przetwarzanie DEX
kod Androida + zasoby + manifest
    ↓ pakowanie i podpis debug
app/build/outputs/apk/debug/app-debug.apk
```

Zależności między zadaniami ustala Gradle. To diagram logiczny, nie obietnica, że wszystkie zadania wykonają się dokładnie sekwencyjnie. Kod wygenerowany przez Room znajduje się w `app/build/generated/ksp/...`; poprawiamy encje i DAO, a nie wygenerowane implementacje.

Profesor pyta: jak zbudować projekt na tym komputerze?

W katalogu `/home/atur/Desktop/MYAPP` istnieje lokalny SDK `/home/atur/Android/Sdk`, z platformą 36 i Build Tools 36.0.0. `java` nie jest domyślnie dostępna w PATH tej powłoki, ale Android Studio dostarcza JBR. Sprawdzona komenda:

```
cd /home/atur/Desktop/MYAPP
JAVA_HOME=/home/atur/.local/share/android-studio/jbr \
  ./gradlew :app:assembleDebug :app:lintDebug --console=plain
```

Na innym komputerze ścieżka JDK i SDK będzie inna. `local.properties` ma zawierać właściwe `sdk.dir=...`; nie należy publikować lokalnych ścieżek jako wspólnej konfiguracji repozytorium. Pierwsze budowanie może potrzebować sieci do pobrania Gradle i bibliotek.

Do analizy grafu:

```
./gradlew :app:dependencies --configuration debugRuntimeClasspath
```

Do release:

```
./gradlew :app:assembleRelease
```

W kodzie nie ma własnej konfiguracji podpisu release. Samo zbudowanie tego wariantu nie tworzy kompletnej procedury produkcyjnej dystrybucji. W tej dokumentacji potwierdzono budowanie debug; nie przeprowadzono publikacji ani instalacji release.

Instalator Linux

Profesor pyta: czy install.sh kompiluje lokalny projekt?

Nie. install.sh pobiera gotowy `Stanowisko-1.0.9.apk` z wydania `android-v1.0.9`. Zmiana lokalnego Kotlinu i uruchomienie tego skryptu nie zainstaluje lokalnej zmiany. Do tego potrzebne są lokalne budowanie i świadome wybranie powstałego APK.

Instalator używa `set -euo pipefail`, tymczasowego katalogu i sprzątania przez `trap`. Pobiera wyłącznie po HTTPS, z limitami czasu i prób. Sprawdza wbudowaną sumę APK oraz, jeśli pobiera ADB, sumę archiwum Platform-Tools 36.0.2 dla Linuksa x86_64.

ADB szuka kolejno w PATH, wskazanym SDK i cache. Pobranie narzędzi wymaga akceptacji warunków. Cache można wskazać `STANOWISKO_CACHE_DIR`. Sam skrypt nie instaluje Android Studio.

Profesor pyta: jak chroni się przed pomyleniem tabletów?

Wymaga dokładnie jednego wpisu urządzenia w `adb devices`, stanu `device`, API co najmniej 34, producenta i

modelu. Dla serialu `emulator-*` dodatkowo sprawdza `ro.kernel.qemu`. Zachowuje serial, a po pobraniu ponownie kontroluje połączenie, model, producenta i właściciela.

To kontrola operacyjna urządzenia wskazanego przez ADB, nie kryptograficzna atestacja jego tożsamości. Skrypt nie wymusza, aby serial oznaczał fizyczne USB; wymaganie USB w README jest instrukcją organizacyjną, podczas gdy logika bazuje na liście ADB.

Profesor pyta: kiedy następuje instalacja i nadanie właściciela?

Najpierw sprawdzenie aktualnego właściciela, pobranie APK, suma i powtórne kontrole. Potem `adb -s SERIAL install -r -t APK` i wymóg potwierdzenia `Success`. `-r` oznacza aktualizację z zachowaniem danych, jeśli system akceptuje zgodność pakietu i podpisu; `-t` dopuszcza pakiet testowy.

Jeśli Stanowisko jest już właścicielem, konfiguracja pozostaje. Inny właściciel albo nierozpoznany stan zatrzymuje procedurę. Brak właściciela prowadzi do pytania `t/N`; po zgodzie wykonywane jest `dpm set-device-owner --user 0 ...`, a wynik zostaje ponownie odczytany. Odmowa pozostawia zainstalowaną aplikację bez pełnej ochrony.

Wejście z `/dev/tty`, `shell -n` i przekierowanie standardowego wejścia dla ADB zapobiegają konfliktowi z uruchamianiem skryptu przez potok. Brak interaktywnego terminala na etapie nadania roli kończy się informacją o niepełnej konfiguracji, a nie domyślną zgodą.

Instalator Windows

Profesor pyta: co jest inne w PowerShell?

install.ps1 implementuje ten sam przebieg, lecz używa .NET i mechanizmów Windows. Wymaga PowerShell 5.1 lub nowszego i platformy Win32NT. Szuka `adb.exe`, korzysta z SDK albo `%LOCALAPPDATA%\Stanowisko`, ewentualnie pobiera oficjalne archiwum Windows 36.0.2. Sprawdza obecność `adb.exe`, `AdbWinApi.dll` i `AdbWinUsbApi.dll`.

`Invoke-Adb` uruchamia proces bez powłoki, zamyka stdin, czyta stdout i stderr asynchronicznie i nakłada timeout: standardowo 30 sekund, dla instalacji 180. Osobna funkcja cytuje argumenty Windows. Blok `finally` usuwa katalog roboczy i przywraca poprzednie ustawienia TLS oraz preferencje PowerShell.

Komenda z README usuwa początkowy BOM przez `.TrimStart([char]0xfeff)` przed zbudowaniem scriptblocku. To zmiana odpowiadająca ostatniemu commitowi. Nie jest to zmiana kodowania APK ani reset systemowych zasad wykonywania skryptów.

Podpis i przygotowanie właściciela

Profesor pyta: dlaczego aktualizacja może zostać odrzucona mimo tej samej nazwy?

Android weryfikuje pakiet i podpis. Lokalny debug może użyć innego klucza niż opublikowane wydanie

rozwojowe. SHA-256 pliku APK sprawdza integralność pobranego pliku, ale nie zastępuje zgodności certyfikatu aktualizacji. Nie należy usuwać istniejącej aplikacji tylko po to, by ominąć ten błąd: usunięcie może oznaczać utratę danych i nie jest częścią instalatora.

Manifest debug ma `testOnly=true`. README deklaruje taki charakter publikowanego APK 1.0.9 i dotychczasowy podpis debug. Własny trwały klucz release i kontrolowana dystrybucja wymagają odrębnego przygotowania. Klucze i ustawienia podpisu są wykluczone z Git.

Profesor pyta: czy Device Owner można nadać na każdym używanym tablecie?

Nie. Android stawia warunki przygotowania urządzenia; konta, profile i istniejące zarządzanie mogą uniemożliwić tę operację. Instalator nie resetuje urządzenia, nie usuwa kont i nie rozwiązuje za użytkownika konfliktów zarządzania. Rola nie jest nadawana samym włączeniem klasycznego administratora w ustawieniach. Projekt nie implementuje provisioningu przez QR. Źródło platformowe: przygotowanie urządzeń dedykowanych.

8. Decyzje techniczne, alternatywy i ograniczenia

Jak rozumieć uzasadnienie projektu

Profesor pyta: czy każde rozwiązanie w istniejącym kodzie jest najlepszym możliwym?

Nie. Projekt jest zbiorem kompromisów dostosowanych do lokalnie zarządzanego stanowiska. Poniżej oddzielamy obserwowaną konstrukcję od jej inżynierskiego uzasadnienia. Dostępna historia nie dokumentuje szczegółowej dyskusji autora o większości tych decyzji; nie traktujemy analizy korzyści jako dowodu jego intencji.

Najważniejsza oś oceny brzmi: czy rozwiązanie daje wymaganą kontrolę nad konkretnym urządzeniem, czy można odtworzyć jego stan po awarii i czy administrator potrafi zdiagnozować niezgodność między zapisem a systemem.

Platforma i architektura

Profesor pyta: dlaczego natywny Android zamiast samej strony lub PWA?

Sterowanie `DevicePolicyManager`, rolą HOME, uprawnieniami innych aplikacji, foreground service i nakładką wymaga integracji z Androidem. Strona WWW może być treścią kiosku, ale nie realizuje sama tych systemowych ról. Natywna warstwa ma więc bezpośredni związek z celem. Kosztem jest związanie z platformą i konieczność osobnej obsługi jej wersji oraz zachowań producentów.

Aplikacja wieloplatformowa również mogłaby mieć moduł natywny realizujący te operacje. Wtedy dodatkowa warstwa miałaby sens przy rzeczywistym współdzieleniu produktu z innymi platformami; nie usuwa potrzeby kodu Androida.

Profesor pyta: dlaczego Device Owner i Lock Task zamiast samego przypięcia ekranu?

Kod potrzebuje zarządzanej listy wielu aplikacji, trwałego HOME i kontroli polityk. Systemowy mechanizm zarządzania odpowiada temu zakresowi. Zwykłe przypięcie jest słabszym, inaczej opuszczanym trybem; projekt jawnie je rozróżnia. Ceną jest trudniejsze przygotowanie urządzenia i większa odpowiedzialność administratora. Szczegóły platformowe wyjaśnia przewodnik Lock Task.

Profesor pyta: dlaczego Kotlin i Compose zamiast Java i układów XML?

Kotlin pasuje do korutyn, Flow i krótkich modeli danych. Compose pozwala opisać UI jako funkcję stanu, co widać przy przełączaniu między pulpitem i administracją. XML Views byłyby równie możliwe, ale wymagałyby innego mechanizmu wiązania stanu i aktualizowania elementów.

Kod sam pokazuje, że nie zastosowano jednej technologii za wszelką cenę: WebView i okno nakładki korzystają z tradycyjnych Views. To naturalne miejsca integracji z istniejącymi API Androida. Kosztem mieszania jest utrzymywanie dwóch sposobów budowania interfejsu.

Profesor pyta: dlaczego jeden moduł i ręczne składanie usług?

Przy 26 plikach i jednym produkcie daje to prosty start i łatwe śledzenie wywołań. Nie wymaga konfiguracji kontenera DI i licznych modułów. Z czasem duże `PolicyEngine` i `WorkspaceScreen` zaczynają łączyć zbyt wiele odpowiedzialności. Wydzielenie adaptera DPM, repozytorium konfiguracji i sterownika sesji byłoby uzasadnione, gdy pojawią się trudności w testowaniu lub nowe warianty polityk.

Sama duża liczba warstw nie poprawia niezawodności. Warto zacząć od granic, które pozwalają testować ważne skutki: „co system zaakceptował", „co zapisano" i „co należy ponowić".

Dane i współbieżność

Profesor pyta: dlaczego Room, a nie wszystko w preferences?

Polityka ma schemat, wersje i powiązane pola, a dziennik wymaga sortowania i retencji. Room daje DAO, walidację zapytań i jawne migracje. Preferences są prostsze dla pojedynczego zestawu ustawień, ale dziennik z operacją „ostatnie 80, przechowuj 1000" naturalnie odpowiada bazie.

Surowe SQLite dałoby pełną kontrolę przy większej ilości ręcznego kodu. Obecne Room wymaga KSP i utrzymywania migracji. Eksporty JSON nie zastępują testu aktualizacji bazy z każdej obsługiwanej wersji.

Profesor pyta: dlaczego lista reguł jest jednym JSON-em?

Silnik zawsze stosuje całą listę jako konfigurację. Jeden dokument pozwala łatwo przechować parę „zapisana lista / zastosowana lista" w pojedynczym rekordzie. Oddzielna tabela reguł byłaby lepsza przy wielu profilach, zapytaniach po pakiecie, relacjach i rozbudowanych regułach. Obecny format jest mniej wygodny do SQL-owej analizy i nie ma własnego pola wersji serializacji.

Profesor pyta: dlaczego SharedPreferences zamiast DataStore?

W obecnym kodzie preferences są prostym magazynem małych wartości, a wynik `commit()` można bezpośrednio sprawdzić. DataStore dałby bardziej jednolity asynchroniczny interfejs i model aktualizacji. Przeniesienie nie zapewni jednak transakcji z DPM i Room. Korzyść pojawi się przy uporządkowaniu przepływów danych, a nie samym zastąpieniu nazw API.

Profesor pyta: dlaczego korutyny i Mutex?

Oczekiwanie na Lock Task lub gotowość nakładki nie musi zajmować wątku, a operacje kryptograficzne i odczyty PackageManager mogą przechodzić na IO. Mutex silnika porządkuje działania administratora. Alternatywą byłby jawny sterownik poleceń lub automat stanów, który łatwiej odtwarza niedokończone operacje. Sam Mutex nie rozwiązuje awarii procesu ani współbieżności wszystkich menedżerów.

Profesor pyta: dlaczego commit, choć lint zaleca apply?

`commit()` daje synchroniczną odpowiedź o zapisie. Dla znacznika sesji lub credential jest to inna potrzeba niż wygodna, asynchroniczna aktualizacja położenia panelu. Zastąpienie wszystkich wywołań przez `apply()` wyłącznie po to, by zniknęło ostrzeżenie, zmieniłoby tę własność. Jednocześnie wywołania dyskowe na głównym wątku mogą kosztować płynność; warto świadomie przenosić je lub zmienić model zapisu. Ostrzeżenie jest sygnałem do oceny kontekstu.

Granice bezpieczeństwa i niezawodności

Profesor pyta: jakie zagrożenia rzeczywiście obejmuje projekt?

Celuje w kontrolę zwykłego korzystania z zarządzanego urządzenia: wybór aplikacji, dostęp do administracji i zakończenie sesji. Z kodu nie wynika pełny model ochrony przed przejętym systemem, rootem, odblokowanym bootloaderem, atakami fizycznymi czy błędami producenta. Nie jest też centralnym MDM z odwoływaniem uprawnień i zewnętrznym audytem.

Poniższe punkty opisują konsekwencje istniejącego kodu lub obszary wymagające testów. Nie są raportem z potwierdzonego ataku na tablet.

Obszar	Dowód w implementacji	Konsekwencja
Początkowy PIN	`DEFAULT_PIN = "1111"`	Przygotowanie urządzenia musi obejmować zmianę znanego poświadczenia
Przejścia sesji	`launch`, `launchWeb`, `endSession` nie mają pełnej kompensacji analogicznej do `apply`	Błąd po zapisie znacznika lub zmianie DPM może pozostawić stan częściowy
Serwis	`enabled=false` zapisywane przed zwolnieniem ograniczeń	Awaria zwalniania może pozostawić ograniczenia bez automatycznego odtwarzania polityki
Przywracanie	Wywoływane z `onResume`, zależy od owner, bazy i keyguardu	Wymaga prób bootu, aktualizacji i utraty procesu na docelowym systemie
Nakładka	Osobna zgoda, foreground service, WindowManager	Dostęp administratora zależy także od dostępności okna i polityki systemu
Lista pakietów	Polityka działa na pakietach	Zaufanie obejmuje także zachowanie i własne ekrany dozwolonych programów
Filtr WWW	Origin i główne żądania	Nie ogranicza globalnie ruchu i wszystkich podzasobów
HTTP	`usesCleartextTraffic=true`	Możliwe połączenie bez szyfrowania, jeśli administrator wybierze taki URL
Dane WWW	Brak czyszczenia cookies i storage przy końcu sesji	Kolejny użytkownik może odziedziczyć stan logowania strony
Uprawnienia	Jawne GRANTED/DENIED bez ogólnej procedury cofnięcia	Decyzje pozostają po serwisie i końcu sesji
Czas blokady PIN	`blockedUntil` używa zegara kalendarzowego	Zmiany czasu wpływają na długość blokady
Rejestr zdarzeń	Lokalna baza, limit 1000, brak podpisu i eksportu	Dziennik służy diagnostyce lokalnej; nie zapewnia niezależnego dowodu zdarzeń

Obszar	Dowód w implementacji	Konsekwencja
Dystrybucja	README opisuje wydanie debug/testOnly, brak signingConfig release	Docelowe utrzymanie wymaga własnej polityki kluczy i aktualizacji
Instalatory	Parser konkretnego wyjścia `dpm list-owners`	Zmiana formatu lub nietypowy system może przerwać instalację

Źródła powyższych zależności: PolicyEngine.kt, WebActivity.kt, PinStore.kt, install.sh, install.ps1.

Profesor pyta: czy udane apply oznacza sprawdzenie absolutnie wszystkiego?

Nie. `setPolicy()` porównuje pakiety, maskę Lock Task i HOME; kolejne kroki potwierdzają LOCKED i gotowość nakładki. Kod nie wykonuje kompletnego audytu wszystkich możliwych ekranów Androida, ustawień OEM ani każdej restrykcji. Podobnie brak wyjątków w systemowym API nie dowodzi oczekiwanego zachowania każdej aplikacji zewnętrznej.

Profesor pyta: co z obsługą błędów i audytem?

`WorkspaceException` daje zamierzony tekst użytkownika, inne wyjątki prowadzą do komunikatu ogólnego. To ogranicza pokazywanie wewnętrznych szczegółów. Równocześnie część operacji audytu jest wykonywana w tej samej ścieżce co działanie właściwe. Błąd bazy przy zapisie logu może utrudnić zgłoszenie pierwotnego błędu albo sprawić, że zakończone działanie wygląda dla użytkownika na nieudane. W przyszłości warto odróżnić wynik operacji od wyniku jej diagnostycznego zapisu.

Silnik i niektóre ekrany łapią szerokie `Exception`, podczas gdy ViewModel osobno ponawia `CancellationException`. Ujednolicenie semantyki anulowania powinno uwzględnić, czy rozpoczęta zmiana systemowa ma zostać dokończona, skompensowana czy zapisana do odtworzenia. Samo mechaniczne dodanie `throw` w każdym miejscu może zmienić bezpieczeństwo przejść.

Kolejność możliwych ulepszeń

Profesor pyta: od czego rozpoczęłabym rozwój, gdyby projekt miał trafić na więcej urządzeń?

1. **Spójność przejść.** Wprowadzić jawne stany operacji: żądana, wykonywana, potwierdzona, wymagająca naprawy. Zapisywać wystarczające dane do wznowienia lub kompensacji po utracie procesu. Szczególnie objąć początek i koniec sesji oraz serwis.
2. **Weryfikacja na urządzeniu.** Zautomatyzować scenariusze restartu, utraty procesu, niegotowych uprawnień, awarii nakładki i aplikacji. Osobno mierzyć wynik systemowy i stan bazy.
3. **Granica API systemu.** Wydzielić adapter DevicePolicyManager i interfejs przechowywania stanu. Ułatwi to testowanie odmów i połowicznie wykonanych operacji.
4. **Poświadczenia i dystrybucja.** Zaplanować wymuszoną zmianę PIN-u początkowego, utrzymanie klucza release, odzyskiwanie dostępu zgodne z celem produktu i kontrolę aktualizacji.
5. **Polityka WWW.** Jawnie ustalić, czy celem jest wyłącznie nawigacja, czy także ograniczenie ruchu i izolacja

użytkowników. Od tego zależą filtrowanie zasobów, obsługa SSO oraz czyszczenie danych strony.

6. **Obsługa floty.** Dopiero przy takim wymaganiu rozważyć provisioning QR, centralne profile, zdalną diagnostykę, eksport audytu i mechanizm aktualizacji.

To propozycje rozwoju, nie funkcje już zrealizowane w obecnym drzewie. Dokumentacja zadania nie zmieniała kodu ochrony ani polityk żadnego urządzenia.

9. Obsługa, diagnostyka i nauka projektu

Przygotowanie stanowiska

Profesor pyta: jak przejść od zainstalowanego APK do rzeczywistej pracy?

Poniższa procedura opisuje czynności administratora. Nie oznacza, że podczas pisania dokumentacji zostały wykonane na podłączonym urządzeniu.

1. Uruchom aplikację i wejdź do administracji PIN-em początkowym `1111`, o ile instalacja nie ma już innego kodu. Zmień PIN i sprawdź jego działanie.
2. Potwierdź rolę Device Owner. Jeśli jej brakuje, przygotuj urządzenie zgodnie z warunkami Androida i instalatorem. Sama rola domyślnego pulpitu nie wystarcza.
3. Zezwól na pływające menu w ustawieniach Androida, korzystając z przycisku w panelu przy wyłączonej ochronie.
4. Przygotuj aplikacje: kreatory pierwszego uruchomienia i ustawienia specjalne przejdź w trybie serwisowym. Dla uprawnień dangerous zapisz jawne zgody lub odmowy w panelu Stanowiska.
5. Wybierz pakiety i tryb każdego z nich. Jeśli potrzebna jest strona, wpisz pełny URL, tytuł i dodatkowe origins logowania; zapisz ustawienia i włącz stronę.
6. Ustal niewygaszanie i widoczność pasków, następnie wybierz „Zapisz konfigurację”.
7. Wybierz „Zastosuj zapisaną politykę”. Sprawdź rewizję zastosowaną, rzeczywisty stan blokady i działanie nakładki nad programami.
8. Przetestuj osobno powrót w wielu aplikacjach, wyjście przez PIN z pojedynczej sesji, odtworzenie po restarcie i świadome wyjście serwisowe.

Źródła odpowiadających przycisków i operacji: WorkspaceScreen.kt, PolicyEngine.kt.

Profesor pyta: jak później zmienić listę bez nieporozumień?

Zakończ aktywną pojedynczą sesję po PIN-ie, otwórz administrację, zmień wybór i zapisz. Zapamiętaj, że `revision` może już być większe od `appliedRevision`; stara lista nadal obowiązuje podczas ochrony. Dopiero zastosowanie zapisanej polityki uruchamia zmianę. Jeśli zmieniły się tylko ustawienia WWW, są zapisywane odrębnie i dotyczą kolejnego utworzenia strony.

Diagnoza po objawie

Profesor pyta: co sprawdzić, zanim zaczniemy zmieniać kod?

Objaw	Pierwsza kontrola	Odpowiedni fragment

Objaw	Pierwsza kontrola	Odpowiedni fragment
Aplikacja działa, ale „Bez pełnej ochrony”	Owner, `enabled`, stan LOCKED; każdy z osobna	`WorkspaceLauncher`, `PolicyEngine.owner/lockState`
Zapisano listę, ale stare kafelki nadal są aktywne	Porównaj rewizję zapisaną i zastosowaną	`save`, wybór `appliedRules` w ekranie
Zastosowanie zgłasza pustą listę	Wybierz aplikację albo poprawnie włącz stronę	`PolicyValidator.validate`
Czerwona obwódka aplikacji	Sprawdź liczbę nieprzygotowanych uprawnień, zapisz zgody lub odmowy	`AppPermission.prepared`, dialog uprawnień
Nie można rozpocząć pojedynczej sesji	Czy działa pełna ochrona i nakładka	`launch`, `launchWeb`
PIN nie reaguje od razu	Czy trwa blokada po próbach, czy operacja jest zajęta	`PinStore.verify`, `busy`
Dostęp administratora wygasł podczas pracy	Termin dwóch minut i callbacki zatrzymania Activity	`authorized`, `onActivityStopped`
Ustawienie uprawnień odrzucone	Zależności lokalizacji i wynik odczytu DPM	`PermissionGrantPlan`, `configure`
Strona odrzuca logowanie	Dokładny schemat, host i port origin przekierowania	`WebAddress.origin`, `page.allows`
HTTPS nie otwiera się	Certyfikat, nazwa hosta, czas i zaufanie urządzenia	`onReceivedSslError` anuluje połączenie
Brak przycisku wyboru pliku lub aparatu WWW	To zaprogramowana odmowa tych funkcji	`WebChromeClient`, download listener
Po restarcie brak gotowej sesji	Keyguard, zapis `enabled`, owner, `policy.error`, dziennik	`BootReceiver`, `restore`
Nakładka poza spodziewaną pozycją	Obrót, insets, zapisane ułamki i aktualne granice	`FloatingPanelBounds`, `restorePosition`
Instalator widzi kilka urządzeń	Pełna lista ADB, także emulatory i połączenia sieciowe	Wybór urządzenia w instalatorach
ADB odrzuca aktualizację	Pakiet, versionCode, podpis i flagi instalacji	`install -r -t` i wynik systemu
Gradle zgłasza brak Java	Ustaw rzeczywistą ścieżkę JDK, np. sprawdzony JBR	Rozdział 7
Rola właściciela odrzucona po instalacji	Odczytaj przyczynę Androida i stan kont/profili	APK może pozostać poprawnie zainstalowany

Profesor pyta: jakie polecenia diagnostyczne nie zmieniają polityki?

Na komputerze z ADB można odczytać:

```
adb devices -l
adb -s SERIAL shell -n getprop ro.build.version.sdk
adb -s SERIAL shell -n dpm list-owners
adb -s SERIAL shell -n dumpsys device_policy
adb -s SERIAL shell -n dumpsys activity activities
```

`SERIAL` jest symbolem do zastąpienia rzeczywistym identyfikatorem. Ostatnie odczyty bywają obszerne i zależne od wersji systemu. Wyniki mogą zawierać dane urządzenia i nazwy uruchamianych aplikacji; przy analizie wybieramy pola potrzebne do problemu. Dziennik w panelu pokazuje 80 ostatnich wpisów, choć baza

przechowuje do 1000.

Profesor pyta: czy do naprawy błędu należy od razu usuwać dane?

Nie wynika to z diagnozy. Najpierw trzeba wiedzieć, czy niespójność dotyczy szkicu, sesji, autoryzacji, nakładki czy DPM. Kasowanie danych utraciłoby PIN, bazę i zapis poprzednich launcherów; nie jest neutralnym sposobem odświeżenia ekranu. Wbudowaną drogą administratora jest zakończenie sesji i wyjście serwisowe, z kontrolą jego faktycznego wyniku.

Scenariusze testów na urządzeniu

Profesor pyta: jak odróżnić pokaz działania od wiarygodnego testu?

Trzeba ustalić stan początkowy, wywołać konkretny bodziec, a następnie odczytać wynik z UI i systemu. Te scenariusze są planem weryfikacji sprzętowej, nie raportem z jej wykonania.

Scenariusz	Bodziec	Oczekiwany wynik i obserwacja
Świeża instalacja	Start bez danych	Domyślny PIN powstaje raz; można go zmienić
Aktualizacja z danymi	Zgodny podpis i nowsza wersja	Własny PIN, baza i konfiguracja pozostają
Szkic przy ochronie	Zapis nowej listy bez zastosowania	Rewizje różne, aktywna pozostaje poprzednia lista
Wieloaplikacyjność	Otwarcie A, dom, otwarcie B	Brak PIN-u do zwykłego powrotu; Lock Task nadal LOCKED
Pojedyncza aplikacja	Próba powrotu i błędny PIN	Sesja pozostaje; poprawny PIN umożliwia zakończenie
Sam powrót z sesji	Użycie przycisku kłódki i poprawny PIN	Pulpit bez otwartego dostępu administratora
Restart w sesji	Restart Androida i odblokowanie keyguardu	Odtworzenie zastosowanej listy i celu; kontrola logu
Utrata procesu	Kontrolowane przerwanie procesu w środowisku testowym	Brak utrwalenia autoryzacji; sprawdzenie ponownego odtworzenia sesji
Nowe uprawnienie programu	Aktualizacja dodająca dangerous permission	Wymagane przygotowanie zamiast swobodnego uruchomienia
Odmowa DPM	Niedopuszczalna zmiana w kontrolowanej próbie	Czytelny błąd, kontrola polityki i możliwej kompensacji
Brak nakładki	Nieprzyznane zezwolenie przed zastosowaniem	Operacja odrzucona przed zakończeniem aktywacji
Awaria nakładki	Kontrolowana awaria usługi podczas pracy	Ocena ścieżki autoryzacji i odtworzenia na danym systemie
Nawigacja WWW	Origin dozwolony, obcy, POST, przekierowania	Odrzucanie niepożądanej nawigacji; osobny pomiar faktycznych żądań sieciowych
Awaria renderera	Kontrolowane zakończenie renderera w próbie	Panel błędu; odtworzenie WebView bez nieautoryzowanego końca sesji
Obrót i mały ekran	Kilka rozdzielczości oraz orientacji	Kontrolki dostępne, panel w granicach, czytelny tekst
Serwis	PIN, koniec sesji i `serviceExit`	NONE, brak nakładki, przywrócone launchery; owner i PIN zachowane

Scenariusz	Bodziec	Oczekiwany wynik i obserwacja
Menu producenta	Przyciski i gesty OEM na docelowym modelu	Sprawdzenie, czy nie otwierają niezamierzonej drogi poza dozwoloną pracę

Profesor pyta: jakie testy automatyczne mają największą wartość przy rozwoju?

Testy migracji 1→3 i 2→3 z rzeczywistymi bazami; testy odmów DPM na poszczególnych etapach; odtwarzanie po utracie procesu pomiędzy zapisem runtime a zmianą polityki; testy poprawnego rozróżnienia PIN-owego powrotu i otwarcia panelu; testy WebView z kontrolowanym serwerem przekierowań i POST. Jednostkowe testy czystych walidatorów są przydatne, ale nie pokrywają tych granic systemowych.

Ćwiczenia z czytania i rozwijania kodu

Profesor pyta: jak samodzielnie prześledzić jedną operację?

Weź przycisk „Zastosuj zapisaną politykę" w `WorkspaceScreen`. Zaznacz kolejno `vm.run`, `engine.apply`, `setPolicy`, `startProtection`, `overlay` i `dao.put`. Dla każdego kroku dopisz: warunki wejścia, odczyty, skutki uboczne i reakcję na błąd. W ten sposób otrzymasz rzeczywisty przebieg, a nie tylko listę klas.

Profesor pyta: jak dodać nowe ustawienie do polityki?

Załóżmy dodatkową opcję o roboczej nazwie `newOption`. Najpierw ustal API Androida i warunki jego użycia. Następnie zdecyduj, czy ustawienie ma obowiązywać dopiero po zastosowaniu. Jeśli tak, potrzebuje pary zapisanej i zastosowanej, migracji bazy, kontrolki szkicu, przekazania przez `save`, obsługi `apply`, `restore` i serwisu oraz zdefiniowanego zachowania kompensacji. Dodaj kontrolę efektu tam, gdzie system pozwala go odczytać.

To ćwiczenie nie implementuje tej opcji. Pokazuje, dlaczego zmiana w projekcie przebiega przez kilka odpowiedzialności i dlaczego zgubiona kopia `applied...` może spowodować użycie szkicu po restarcie.

Profesor pyta: jak ocenić propozycję nowej biblioteki?

Nazwij problem, który rozwiązuje, porównaj go z obecnym kodem i dopiero potem koszt zależności. Biblioteka HTTP nie jest potrzebna tylko dlatego, że produkt otwiera URL: obecny ruch strony realizuje WebView. Biblioteka DI ma sens, gdy poprawia składanie i testowanie, a nie dlatego, że sama jej obecność miałaby dowodzić dobrej architektury.

Pytania na obronę projektu

Profesor pyta: dlaczego nie utożsamiamy enabled z pełną ochroną?

Bo to trwały zamiar aplikacji; skuteczne działanie wymaga także roli owner i potwierdzonego stanu Androida. Te wartości mogą rozjechać się po błędzie.

Profesor pyta: czy Mutex zabezpiecza przed restartem?

Nie. Porządkuje operacje w żyjącym procesie. Restart wymaga trwałego zapisu i logiki odtwarzania.

Profesor pyta: czy jawna odmowa uprawnienia jest błędem przygotowania?

Nie. Własność `prepared` traktuje administracyjne DENIED jako rozstrzygnięcie. Program może mieć mniej funkcji, ale nie musi pytać użytkownika o tę zgodę.

Profesor pyta: dlaczego systemowy HOME może być wyłączony nawet w trybie wielu aplikacji?

Bo maska dodaje HOME tylko przy `single=false` i `hideBars=false`. Dla ukrytych pasków powrót zapewnia nakładka.

Profesor pyta: czy serwis usuwa zarządzanie urządzeniem?

Nie. Zdejmuje określone ograniczenia, pozostawiając Device Owner, zapisane dane i decyzje uprawnień.

Profesor pyta: czy release i debug różnią się tylko nazwą pliku?

Nie. Wariant debug scala dodatkowy manifest `testOnly`; podpisywanie i przeznaczenie dystrybucji też mają znaczenie. W tym repozytorium brak gotowej konfiguracji własnego podpisu release.

Profesor pyta: jaka jest najważniejsza rzecz do wykazania przy prezentacji?

Pokazać pełny przebieg: przygotowanie, zapis, zastosowanie, uruchomienie, kontrolowany powrót i odtworzenie. Sam ładny pulpit nie dowodzi działania egzekwowania polityki, a sam numer rewizji nie dowodzi stanu systemu.

10. Źródła, weryfikacja i słownik

Podstawa opisu

Profesor pyta: na czym opierają się twierdzenia tego podręcznika?

Na kodzie i konfiguracji w `/home/atur/Desktop/MYAPP`, gałąź `android`, HEAD `bb7dd3cf5de9124162a05444844600cced8a0a40`. Opisanym wydaniem jest `versionName=1.0.9`, `versionCode=11`. Analiza obejmuje wszystkie 26 własnych plików Kotlin, oba manifesty, zasoby, trzy schematy bazy, konfigurację Gradle i oba instalatory.

Odnośniki do plików w rozdziałach wskazują dowody implementacyjne. Uzasadnienia i propozycje rozwoju są nazwane jako analiza inżynierska. Historia Git pozwala ustalić pochodzenie lokalnej kopii i pięć zmian opisanych w rozdziale 1, ale nie pełną historię powstania pomysłu.

Wyniki kontroli lokalnych

Profesor pyta: co faktycznie uruchomiono 6 listopada 2026 roku?

Kontrola	Wynik	Co potwierdza
`:app:assembleDebug` i `:app:lintDebug`	`BUILD SUCCESSFUL`, 47 wykonanych zadań, 1 min 6 s	Aktualny kod daje zbudować debug APK w opisanym środowisku; lint ukończony
Raport lint XML	0 błędów, 28 ostrzeżeń	Zakres statycznej analizy debug, bez usunięcia wskazanych ostrzeżeń
Graf `debugRuntimeClasspath`	Odczyt zakończony sukcesem	Faktycznie rozwiązane m.in. Kotlin stdlib 2.2.10, Material3 1.4.0, Compose UI 1.10.0
`bash -n install.sh`	Sukces	Poprawność składni Bash, bez wykonywania instalacji
Metadane APK przez `aapt2`	Pakiet `pl.adriantur.stanowisko`, wersja 11/1.0.9, min 34, target 36	Zawartość zbudowanego artefaktu odpowiada konfiguracji
`apksigner verify --print-certs`	Podpis zweryfikowany, certyfikat `CN=Android Debug`	Integralność podpisu lokalnego APK; nie zgodność z podpisem pobranego wydania
Próby na skompilowanych klasach	62 sprawdzenia zakończone sukcesem	Opisane niżej kontrakty czystej logiki
Kontrola wektora PBKDF2 niezależnym narzędziem	Wyniki zgodne	Zgodność jednego przykładu wyliczenia SHA-256/ PBKDF2
Dokumentacja w przeglądarce Firefox	Sprawdzone wyszukiwanie, brak wyników, reset i przejścia do rozdziałów	Zachowanie nawigacji HTML i 3 osadzone SVG
Układ HTML i druk	Podgląd szeroki 1440 px, wąski 500 px, eksport A4	Brak poziomego przewijania całej strony w tych widokach; pełna treść w PDF

Kontrola	Wynik	Co potwierdza
Scenariusz Adrian_Tur_1.pptx	47 kolejnych slajdów; pełna i krótka wypowiedź, przejście oraz pytanie z odpowiedzią do każdego	Pokrycie prezentacji i zgodność numeracji z kolejnością PPTX
Kopia PPTX z notatkami	Zmieniono tylko 47 plików notesSlide; pozostałe 170 elementów archiwum są identyczne bajtowo	Zachowane slajdy i pierwotne notatki, oryginalny plik bez zmian; kontrola struktury XML, bez uruchamiania PowerPoint

Środowisko: Linux, JetBrains Runtime 25.0.4 z lokalnego Android Studio, SDK `/home/atur/Android/Sdk`, platforma 36, Build Tools 36.0.0 i wrapper Gradle 9.1.0. Wynik APK: `app/build/outputs/apk/debug/app-debug.apk`, 28 932 234 bajty. Raporty narzędzia pozostają w ignorowanym przez Git katalogu `app/build/reports/`.

Lint zgłosił 16 sugestii `UseKtx`, 3 ostrzeżenia `ApplySharedPref` i 9 informacji o nowszych wersjach zależności lub narzędzi. Kompilator dodatkowo zgłosił dwa użycia przestarzałego API `PermissionInfo` w `AppPermissions.kt`. Pakowanie pozostawiło bibliotekę `libandroidx.graphics.path.so` bez usunięcia symboli. Ostrzeżenia nie zatrzymały budowania; dokumentacja nie aktualizowała bibliotek ani nie zmieniała logiki aplikacji.

Profesor pyta: co obejmowały 62 sprawdzenia?

Próby wykonały metody rzeczywiście skompilowanych klas z debug `classes.jar`, z Kotlin stdlib i wygenerowanymi identyfikatorami zasobów. Obejmowały format PIN-u, progi opóźnień, długość pochodnej i czyszczenie tablicy znaków, normalizację origin i błędne URL, zgodę na ścieżki w tym samym origin, odrzucenie obcego schematu/portu/hosta, walidację konfiguracji WWW, pozycjonowanie panelu, walidację listy polityki i kolejność zależności uprawnień lokalizacji.

Przykład kontrolny: PIN `1234`, 32 bajty zerowej soli, 600 000 iteracji, 32 bajty wyniku dały hex `9492d8382d29215db911d4e94fcabff0da4bcda30553deba4512d24f7f2508b6`. Niezależne obliczenie przez Python `hashlib.pbkdf2_hmac` dało tę samą wartość. To **wektor testowy**, nie dane poświadczenia urządzenia. Tymczasowy program sprawdzający został usunięty po wykonaniu; nie dodano do produktu nowego frameworka testowego.

Granice weryfikacji

Profesor pyta: czego te wyniki nie dowodzą?

Nie przeprowadzono w tym zadaniu instalacji ani zmian DPM na fizycznym urządzeniu. Nie potwierdzono na tablecie nadania właściciela od zera, zachowania OEM, odtwarzania po restarcie, szczelności pojedynczej sesji, ruchu WebView ani pracy nakładki. Wymagane próby sprzętowe są rozpisane w rozdziale 9.

W repozytorium nie ma katalogów `app/src/test` i `app/src/androidTest` ani deklaracji zależności testowych. Zadania lint zawierające nazwy AndroidTest/UnitTest nie oznaczają, że takie testy wykonano. README deklaruje wcześniejsze 46 testów instalatora w PowerShell 7 na Linuksie; ich źródła i powtarzalny zestaw nie znajdują się w

analizowanym drzewie, więc ten wynik pozostaje informacją historyczną z README. Nie został odtworzony w tej analizie. Nie uruchomiono instalatora na Windows.

Nie pobierano publikowanego APK w celu dowiedzenia jego identyczności z lokalnym buildem. Instalatory wskazują konkretne wydanie i sumę, natomiast to nie dowodzi samo w sobie, że binarium zostało zbudowane z dokładnie tego commitu w kontrolowanym procesie. Nie ma tu konfiguracji CI ani udokumentowanego odtwarzalnego wydania bit po bicie. Nie znaleziono też pliku licencji, więc sam dostęp do repozytorium nie jest kompletną informacją o warunkach ponownego wykorzystania kodu.

Źródła platformowe

Profesor pyta: gdzie sprawdzać kontrakty Androida zamiast polegać na skrótach?

Źródło pierwotne	Do czego służy w tym opisie
Android Lock task mode	Lista pakietów, LOCKED kontra przypięcie, start i zatrzymanie trybu
Dedicated devices cookbook	Przygotowanie DPC i roli właściciela
AGP 9.0 i 9.0.1	Minimalne narzędzia, wbudowany Kotlin i zgodność
Wbudowany Kotlin	Brak potrzeby osobnego kotlin-android w AGP 9
Gradle 9.1 release notes	Obsługa Java 25 przez tę wersję Gradle
Room migrations	Migracje i zachowanie danych
Android Keystore	Charakter zarządzania kluczami i zależność od sprzętu
WebViewClient	Semantyka callbacków, POST i ograniczenia przekierowań
DevicePolicyManager	Warunki i ograniczenia operacji zarządzania, w tym uprawnień

Źródła sprawdzono przy opracowaniu 6 listopada 2026 r. Ustalenia o tym, **co wywołuje aplikacja**, wynikają z lokalnego kodu; strona platformowa opisuje kontrakt API i nie jest dowodem wykonania go na danym tablecie.

Słownik

Profesor pyta: które pojęcia muszą być precyzyjne?

Pojęcie	Znaczenie w projekcie
APK	Pakiet instalacyjny aplikacji Android
ADB	Narzędzie komunikacji komputera z urządzeniem lub emulatorem
Device Owner	Systemowa rola właściciela zarządzanego urządzenia; nie PIN użytkownika
DPC	Device Policy Controller, aplikacja sterująca politykami
DPM	DevicePolicyManager, API Androida do tych polityk

Pojęcie	Znaczenie w projekcie
Lock Task	Systemowe ograniczenie dostępnych zadań do listy pakietów
Launcher i HOME	Interfejs uruchamiania aplikacji i rola pulpitu Androida
Activity	Ekran lub komponent interakcji Androida
Task	Stos aktywności; nie to samo co proces i nie to samo co pakiet
Foreground service	Usługa zgłoszona systemowi z widocznym powiadomieniem
Overlay	Okno nad innymi aplikacjami, wymagające osobnej zgody
Origin	Schemat, host i efektywny port URL, bez ścieżki
Room i DAO	Warstwa nad SQLite i interfejs operacji na bazie
Flow i StateFlow	Strumień wartości i strumień bieżącego stanu
Coroutine	Zadanie asynchroniczne, które może zawieszać wykonanie
Mutex	Ochrona przed równoczesnym wykonywaniem chronionego fragmentu
PBKDF2	Wyprowadzanie pochodnej z hasła/PIN-u, soli i liczby iteracji
AES-GCM	Szyfrowanie uwierzytelnione zapewniające poufność i kontrolę integralności
Keystore	Systemowy magazyn i mechanizm wykonywania operacji kluczem
BOM	Zestaw ograniczeń wersji bibliotek; nie własna biblioteka UI
KSP	Przetwarzanie symboli w czasie budowania, tutaj dla Room
Rewizja	Kolejny zapis konfiguracji, odrębny od numeru wydania aplikacji

Utrzymywanie dokumentacji

Profesor pyta: co trzeba zmienić w podręczniku po zmianie kodu?

Zaktualizuj commit i datę, a następnie rozdział odpowiadający zachowaniu. Nowa kolumna wymaga sprawdzenia modelu danych, migracji i wszystkich ścieżek zastosowania. Nowa zależność wymaga aktualizacji tabeli bezpośrednich bibliotek i odczytu rozwiązanej wersji. Zmiana polityki WWW wymaga poprawienia przykładów origin i scenariuszy nawigacji. Zmiana instalatora wymaga ponownego sprawdzenia jego warunków oraz zgodności Linux/Windows.

Wersja HTML jest pochodną Markdown. generuj_przewodnik.py łączy indeks i numerowane rozdziały, osadza SVG i dodaje wyszukiwanie oraz style. Tworzy też osobny scenariusz prezentacji i krótką ściągę z wariantów wypowiedzi w rozdziale 11. Używa wyłącznie biblioteki standardowej Pythona i lokalnego programu `markdown` (Discount). Nie jest zależnością aplikacji Android. Regeneracja z katalogu projektu:

```
python3 docs/generuj_przewodnik.py
```

Kopię prezentacji z wypowiedziami w notatkach prelegenta można odtworzyć z oryginalnego PPTX. Polecenie

zapisuje osobny plik w `docs`, zachowując źródłową prezentację:

```
python3 docs/generuj_przewodnik.py --pptx /home/atur/Downloads/Adrian_Tur_1.pptx
```

Na tym komputerze Discount 3.0.0d jest dostępny jako `/usr/bin/markdown`. Sam plik wynikowy nie wymaga Pythona, Discount, serwera ani sieci. PDF jest eksportem odpowiedniego HTML; po zmianach odśwież `PRZEWODNIK.pdf`, `SCENARIUSZ_PREZENTACJI.pdf` i `SCIAGA_PREZENTACJI.pdf`, używając „Drukuj / PDF” w przeglądarce. Plik `SCIAGA_PREZENTACJI.md` również powstaje z generatora; poprawki wypowiedzi wprowadzaj w rozdziale 11. Diagramy SVG znajdują się w `docs/diagramy`; ich edycja nie wymaga zewnętrznej usługi. Przed przekazaniem kolejnej wersji sprawdź odnośniki, kompletność mapy plików i faktyczny wynik narzędzi, zamiast przenosić wcześniejszy raport jako aktualny.

11. Co mówić podczas prezentacji Stanowiska

Scenariusz odpowiada dokładnie **47 slajdom** pliku `/home/atur/Downloads/Adrian_Tur_1.pptx`, w kolejności zapisanej w prezentacji. Zawiera gotowe wypowiedzi w pierwszej osobie, krótkie warianty, przejścia i odpowiedzi na pytania profesora. Możesz uczyć się z całego modułu, a podczas wystąpienia korzystać tylko ze skrótu.

Wygodny widok: scenariusz HTML. **Do wydruku:** scenariusz PDF. **Na ostatnią powtórkę:** ściąga PDF.

Do prowadzenia pokazu: kopia PPTX z wypowiedziami w notatkach prelegenta. Slajdy mają zachowaną treść i kolejność; rozszerzone notatki są przeznaczone do widoku prowadzącego.

Jak korzystać ze scenariusza

Powiedz to wypowiedź do wygłoszenia, a nie opis tego, jak należałoby mówić. **Wersja krótka** zastępuje dłuższą wypowiedź, gdy masz mniej czasu. Nie czytaj obu wariantów jeden po drugim. **Przejście** łączy slajd z następnym. **Profesor pyta** i **Odpowiedź** służą przygotowaniu do dyskusji; nie są częścią podstawowego wystąpienia.

Przyjmij roboczo około **20–25 minut na wersję podstawową**, bez pełnej demonstracji na urządzeniu i bez pytań. To szacunek do sprawdzenia stoperem, nie zmierzony czas Twojej wypowiedzi. Przy krótszym limicie korzystaj z wariantów krótkich, a szczegóły techniczne rozwijaj po pytaniu. Demonstrację ze slajdu 39 zaplanuj osobno; restart tabletu ma zmienny czas.

Główna myśl całej prezentacji

Zapamiętaj ciąg: **problem urządzenia zadaniowego → konfiguracja administratora → polityki Androida → trwały stan → weryfikacja na urządzeniu**. Najczęściej wracające rozróżnienia to: instalacja i ochrona, zapis i zastosowanie, obecność kodu i wynik testu, lista nawigacji i filtr całej sieci.

Podstawa wypowiedzi

Tekst slajdów i notatki prelegenta odczytano z podanego PPTX. Twierdzenia o implementacji porównano z `/home/atur/Desktop/MYAPP`, commit `bb7dd3cf5de9124162a05444844600cced8a0a40`. Dane uczelni, tematu i osób oraz przywołania karty pracy pochodzą z prezentacji; sama karta nie jest załącznikiem do tego modułu. Oryginalnego PPTX nie zmieniano.

Prezentacja opisuje stan z 5 listopada 2026 r. Wcześniejsza dokumentacja zawiera dodatkowe sprawdzenia lokalne z 6 listopada: udane budowanie, lint i 62 sprawdzenia wybranej logiki. Możesz o nich powiedzieć jako o **dodatkowym zakresie weryfikacji**, opisanym w rozdziale 10. Nie zastępują prób ochrony na fizycznym tablecie.

Gdzie wrócić po wyjaśnienia

Slajdy	Temat do powtórzenia
1–18	Cel projektu, decyzje i ograniczenia
19–24, 29–31, 40	Architektura, dane, budowanie
25–27, 32–33, 37	Zapis, zastosowanie, sesje i odtwarzanie
34–36	PIN i uprawnienia, WebView
38–47	Procedury i próby, wyniki weryfikacji

Slajd 01 — STANOWISKO

Cel: Przedstaw temat i od razu powiedz, jaki problem rozwiązuje aplikacja.

Powiedz:

> Dzień dobry, nazywam się Adrian Tur. Przedstawię projekt aplikacji mobilnej Stanowisko, która tworzy kontrolowane środowisko pracy na Androidzie. Administrator określa dostępne aplikacje i stronę internetową, a systemowe polityki ograniczają korzystanie z urządzenia do przygotowanego zakresu. W prezentacji pokażę założenia, najważniejsze mechanizmy implementacji oraz sposób weryfikowania ich działania.

Wersja krótka:

> Przedstawiam Stanowisko — aplikację Android do udostępniania wybranych programów i strony WWW pod kontrolą administratora.

Przejście: Najpierw krótko pokażę układ prezentacji.

Profesor pyta: Co oznacza „zdefiniowany komponent” w tym projekcie?

Odpowiedź: W implementacji chodzi przede wszystkim o wybraną aplikację identyfikowaną pakietem oraz wbudowaną stronę WWW. Nie deklaruję filtrowania każdej funkcji i każdego ekranu wewnątrz aplikacji zewnętrznej.

Slajd 02 — Trzy etapy projektu

Cel: Uporządkuj oczekiwania słuchacza.

Powiedz:

> Prezentację podzieliłem na trzy części. Najpierw wyjaśnię problem, zakres i wymagania. Następnie przejdę do implementacji: pulpitu, polityk Androida, danych oraz PIN-u. Na końcu pokażę scenariusz demonstracji, stan weryfikacji i dalsze kroki potrzebne przed wdrożeniem.

Wersja krótka:

> Omówię założenia, implementację i weryfikację, a na końcu kierunki rozwoju.

Przejście: Zacznę od tego, dlaczego zwykły dostęp do Androida bywa niewłaściwy dla stanowiska zadaniowego.

Profesor pyta: Która część jest głównym rezultatem pracy?

Odpowiedź: Aplikacja mobilna z lokalną konfiguracją i implementacją polityk urządzenia. Plan badań i procedura wdrożenia pozwalają następnie ocenić jej działanie w konkretnym środowisku.

Slajd 03 — Urządzenie do konkretnego zadania

Cel: Wyjaśnij problem, zanim wymienisz technologie.

Powiedz:

> Tablet przeznaczony do jednego zadania nie zawsze powinien udostępniać użytkownikowi cały system. Możliwość uruchamiania dowolnych aplikacji albo zmiany ustawień może zakłócać pracę stanowiska. Przykładem jest ekran informacyjny, który powinien stale prezentować wskazaną treść. Problemem projektowym jest więc takie ograniczenie środowiska, aby użytkownik miał potrzebne funkcje, a administrator zachował możliwość obsługi urządzenia.

Wersja krótka:

> Urządzenie zadaniowe powinno udostępniać potrzebne funkcje i utrzymywać konfigurację ustaloną przez administratora.

Przejście: Odpowiedzią jest połączenie listy dozwolonych komponentów z mechanizmami systemowymi.

Profesor pyta: Czy ukrycie ikon rozwiązałoby ten problem?

Odpowiedź: Sam wygląd pulpitu nie egzekwuje ograniczeń systemowych. Potrzebny jest mechanizm kontrolujący zadania i dostępne pakiety, dlatego aplikacja korzysta z polityk Androida.

Slajd 04 — Administrator ustala granice

Cel: Powiąż trzy elementy rozwiązania z problemem.

Powiedz:

> Koncepcja składa się z trzech części. Administrator wybiera dostępne aplikacje i ewentualną stronę. Device Owner i Lock Task pozwalają zastosować ograniczenia w Androidzie. PIN chroni zmianę konfiguracji i autoryzowany powrót z sesji pojedynczej. Istotna jest kolejność: samo zainstalowanie APK i zapisanie listy nie uruchamia jeszcze pełnej ochrony. Politykę trzeba osobno zastosować.

Wersja krótka:

> Administrator wybiera zakres, Android egzekwuje politykę, a PIN chroni administrację. Zapis konfiguracji i jej zastosowanie są osobnymi krokami.

Przejście: Na tej koncepcji opierają się cel pracy i pięć tez.

Profesor pyta: Czy Device Owner i PIN to ten sam rodzaj uprawnienia?

Odpowiedź: Nie. Device Owner jest rolą nadawaną przez Androida. PIN jest mechanizmem autoryzacji wewnątrz Stanowiska, który pozwala korzystać z funkcji administratora aplikacji.

Slajd 05 — Cel pracy i pięć tez

Cel: Przedstaw tezy jako przedmiot sprawdzenia.

Powiedz:

> Celem jest przygotowanie środowiska z dostępem do zdefiniowanych komponentów. Pięć tez dotyczy skuteczności ograniczeń, bezpieczeństwa i przewidywalności, elastyczności listy, użycia oficjalnych API oraz nakładu pracy administratora. Implementacja pokazuje, jak te założenia realizować. Żeby uznać tezy za potwierdzone, potrzebne są jeszcze odpowiednie próby i pomiary, do których wrócę w końcowej części.

Wersja krótka:

> Tezy obejmują kontrolę środowiska, bezpieczeństwo, elastyczność, oficjalne API i wygodę administracji. Ich skuteczność wymaga weryfikacji.

Przejście: Najpierw pokażę architekturę, która ma umożliwiać realizację tych założeń.

Profesor pyta: Czy samo istnienie kodu potwierdza tezę o bezpieczeństwie?

Odpowiedź: Potwierdza obecność zaprojektowanych mechanizmów. Ich skuteczność wymaga prób określonych zagrożeń, np. nieautoryzowanego opuszczenia stanowiska, na przygotowanym urządzeniu.

Slajd 06 — Lokalny obieg konfiguracji

Cel: Przeprowadź słuchacza po schemacie w kolejności przepływu.

Powiedz:

> Zmiana zaczyna się w panelu administratora. ViewModel udostępnia stan i obsługuje akcje interfejsu, a PolicyEngine wykonuje operacje dotyczące polityki. Silnik współpracuje z Androidem oraz lokalnym magazynem danych. Room przechowuje politykę i audyt, a pozostałe ustawienia trafiają do preferences. Ten obieg jest lokalny. Osobną zależność od sieci ma treść wyświetlana przez WebView.

Wersja krótka:

> Interfejs przekazuje akcje do logiki polityk, która współpracuje z Androidem i lokalnymi danymi. Sieci potrzebuje wybrana strona WWW.

Przejście: Zanim przejdę do zastosowań, wskażę źródła wiedzy wykorzystane w prezentacji.

Profesor pyta: Czy jest to architektura klient–serwer?

Odpowiedź: Lokalna konfiguracja i polityki nie wymagają serwera aplikacji. WebView jest klientem skonfigurowanej strony, ale nie oznacza to istnienia centralnego backendu zarządzającego Stanowiskiem.

Slajd 07 — Źródła i bibliografia

Cel: Wyjaśnij rolę źródeł, bez czytania wszystkich pozycji.

Powiedz:

> Bibliografia obejmuje publikacje dotyczące Androida, bezpieczeństwa i Kotlina, a także wskazane w prezentacji prace inżynierskie. Osobną grupą są materiały Android Developers dotyczące kontrolera polityk i urządzeń dedykowanych. Książki służą uporządkowaniu podstaw, natomiast szczegóły działania konkretnych API trzeba odnosić do dokumentacji platformy i wersji użytej w projekcie.

Wersja krótka:

> Podstawą są publikacje o Androidzie i Kotlinie oraz dokumentacja Android Developers dotycząca polityk urządzenia.

Przejście: Przejdę teraz od podstaw do przykładowych zastosowań aplikacji.

Profesor pyta: Czy pozycja o bezpieczeństwie z 2014 roku wystarcza do opisu współczesnego Androida?

Odpowiedź: Może stanowić źródło podstaw architektury. Aktualne kontrakty DevicePolicyManager, Lock Task czy WebView trzeba sprawdzać w dokumentacji odpowiednich API. Nie należy przypisywać starszej książce opisu nowych funkcji.

Slajd 08 — Jedna aplikacja, różne stanowiska

Cel: Pokaż trzy zastosowania tego samego mechanizmu.

Powiedz:

> Ta sama konstrukcja może obsłużyć różne zadania. W kiosku informacyjnym udostępniam stronę. Na tablecie służbowym przygotowuję zestaw aplikacji. Na stanowisku demonstracyjnym uruchamiam jedną aplikację, a powrót chronię PIN-em. Są to scenariusze użycia wynikające z konfiguracji, a nie trzy osobno napisane produkty ani lista zakończonych wdrożeń.

Wersja krótka:

> Zmieniając konfigurację, mogę przygotować kiosk WWW, zestaw aplikacji służbowych albo demonstrację jednej aplikacji.

Przejście: Takie scenariusze trzeba następnie sprawdzić w konkretnym środowisku pilotażowym.

Profesor pyta: Co decyduje o scenariuszu: nowy APK czy konfiguracja?

Odpowiedź: W tych przykładach przede wszystkim lista pakietów, konfiguracja strony i tryb uruchamiania. Nowa funkcja wykraczająca poza te możliwości wymagałaby zmiany kodu.

Slajd 09 — Pilotaż w środowisku testowym

Cel: Określ realny status miejsca wdrożenia.

Powiedz:

> Jako możliwe środowisko pilotażu prezentacja wskazuje uczelnię lub lokalne laboratorium. Pilotaż powinien ustalić docelowy model urządzenia, wersję Androida i zasady organizacji. Wtedy można ocenić zachowanie ochrony oraz wygodę obsługi. Ten slajd opisuje proponowane miejsce prób; nie jest informacją o zakończonym wdrożeniu produkcyjnym w uczelni.

Wersja krótka:

> Pilotaż planuję w środowisku testowym, po określeniu modelu urządzenia i zasad jego użycia.

Przejście: Żeby taki pilotaż był wykonalny, trzeba jasno określić zakres informatyczny pracy.

Profesor pyta: Jaki tablet został już zatwierdzony do wdrożenia?

Odpowiedź: Prezentacja nie potwierdza zatwierdzenia konkretnego modelu. Dobór sprzętu i sprawdzenie jego zachowania są elementami pilotażu.

Slajd 10 — Zakres informatyczny

Cel: Oddziel zadania projektu od przyszłych rozszerzeń.

Powiedz:

> Zakres obejmuje natywne programowanie Androida, interfejs Compose, lokalne dane, polityki urządzenia i kontrolę dostępu. Ważna jest również czytelność konfiguracji oraz przygotowanie scenariuszy prób. Centralne zarządzanie flotą pozostaje osobnym etapem. Dzięki temu projekt skupia się na działaniu pojedynczego stanowiska i jego integracji z systemem.

Wersja krótka:

> Zakres to aplikacja Android, lokalna konfiguracja i polityki urządzenia. Centralne zarządzanie flotą pozostaje poza bieżącą implementacją.

Przejście: Konkretnym rezultatem tego zakresu jest aplikacja mobilna.

Profesor pyta: Dlaczego nie dołączono od razu panelu centralnego?

Odpowiedź: Wymagałby dodatkowo serwera, komunikacji, uwierzytelnienia urządzeń i obsługi synchronizacji. To odrębny zakres, który nie jest potrzebny do przedstawienia lokalnego mechanizmu stanowiska.

Slajd 11 — Dziełem jest aplikacja mobilna

Cel: Nazwij rezultat i jego części.

Powiedz:

> Głównym rezultatem jest lokalnie konfigurowana aplikacja Android. Łączy pulpit użytkownika, panel administratora, dwa sposoby pracy i trwały zapis konfiguracji. Dokumentacja oraz instalatory wspierają przygotowanie urządzenia. Ewentualny panel webowy stanowi osobny komponent; omawiana tutaj aplikacja mobilna nie zawiera centralnego API zarządzania.

Wersja krótka:

> Rezultatem jest aplikacja mobilna z pulpitem, politykami, PIN-em, danymi lokalnymi i instrukcją przygotowania urządzenia.

Przejście: Z tych funkcji korzystają osoby o różnych odpowiedzialnościach.

Profesor pyta: Czy strona otwierana w WebView jest panelem administratora?

Odpowiedź: Nie musi nim być. Jest treścią wskazaną w konfiguracji. Administrację Stanowiska realizuje lokalny interfejs chroniony PIN-em.

Slajd 12 — Użytkownicy i odpowiedzialność

Cel: Wyjaśnij role organizacyjne i ich odpowiedniki w aplikacji.

Powiedz:

> Administrator przygotowuje listę komponentów, PIN i politykę. Operator korzysta z udostępnionych funkcji. Organizacja ustala przeznaczenie urządzenia, a dział IT odpowiada za sprzęt i aktualizacje. W samej aplikacji nie są to cztery rodzaje kont w bazie. Praktyczny podział dostępu polega na zwykłej obsłudze pulpitu i czasowej autoryzacji administratora przez PIN.

Wersja krótka:

> Operator pracuje na pulpicie, administrator ustala konfigurację, a organizacja i IT odpowiadają za zasady oraz przygotowanie sprzętu.

Przejście: Następnie ocenię, czy projekt jest wykonalny przy przyjętych narzędziach i warunkach.

Profesor pyta: Gdzie przechowywane są konta administratorów?

Odpowiedź: Ten projekt nie ma bazy kont i przypisanych ról użytkowników. Weryfikuje jedno lokalne poświadczenie PIN, a autoryzacja administratora jest krótkotrwałym stanem procesu.

Slajd 13 — Wykonalność projektu

Cel: Uzasadnij możliwość realizacji i właściwie odczytaj wartość „0 zł”.

Powiedz:

> Wykonalność techniczna opiera się na Kotlinie, Compose, Room i oficjalnych API Androida. Minimalna obsługiwana wersja to Android 14. Emulator pomaga w rozwoju, natomiast ochronę trzeba sprawdzić na sprzęcie. Wartość zero złotych odnosi się na slajdzie do opłat za wybrany stos narzędzi. Nie jest budżetem całego przedsięwzięcia, bo pozostają koszty urządzenia, przygotowania, testów i pracy.

Wersja krótka:

> Stos pozwala zrealizować projekt na Androidzie 14+. „0 zł” nie obejmuje sprzętu ani nakładu pracy i prób.

Przejście: Realizację podzielono na sześć następujących po sobie etapów.

Profesor pyta: Czy aplikacja nie generuje żadnych kosztów wdrożenia?

Odpowiedź: Ten slajd tego nie wykazuje. Wdrożenie wymaga wyceny sprzętu, pracy, utrzymania i prób. Kwota na slajdzie nie jest analizą całkowitego kosztu.

Slajd 14 — Sześć etapów realizacji

Cel: Pokaż zależność prac zamiast odczytywać listę.

Powiedz:

> Kolejność prac wynika z zależności między częściami aplikacji. Najpierw określam wymagania i dane, następnie buduję pulpit i integruję polityki Androida. Dopiero wtedy można oceniać zachowanie ograniczeń i restartu. Ostatni etap obejmuje pakiet instalacyjny oraz instrukcję. Jest to plan etapów, bez przypisywania im nieudokumentowanych dat czy pracochłonności.

Wersja krótka:

> Najpierw analiza i dane, potem pulpit oraz polityki, a następnie próby i przygotowanie dystrybucji.

Przejście: Przy takiej realizacji trzeba przewidzieć najważniejsze ryzyka.

Profesor pyta: Czy wszystkie etapy są już zakończone?

Odpowiedź: Kod i materiały instalacyjne są dostępne. Pełne próby ochrony, pomiary i ocena wdrożeniowa na docelowym sprzęcie pozostają odrębnym zadaniem.

Slajd 15 — Ryzyka wymagają prób na sprzęcie

Cel: Wybierz najważniejsze ryzyka i wskaż sposób ich ograniczania.

Powiedz:

> Największe ryzyka dotyczą przygotowania Device Owner, różnic producentów i możliwości nieautoryzowanego opuszczenia środowiska. Odpowiedzią są kontrole warunków wejścia oraz próby na docelowym urządzeniu. Pozostałe ryzyka obejmują konfigurację i aktualizację danych. Oceny „średnie” czy „wysokie” są tutaj jakościową oceną projektową; nie pochodzą z policzonej częstości awarii.

Wersja krótka:

> Główne ryzyka to przygotowanie urządzenia, różnice OEM i skuteczność ograniczeń. Ich ocenę trzeba uzupełnić próbami sprzętowymi.

Przejście: Z ryzyk wynikają granice obecnego rozwiązania.

Profesor pyta: Na jakiej próbie oszacowano prawdopodobieństwo jako średnie?

Odpowiedź: Nie jest to estymacja statystyczna. Tabela porządkuje ryzyka jakościowo, żeby wskazać priorytety sprawdzenia. Dane ilościowe wymagałyby określenia urządzeń, scenariuszy i liczby powtórzeń.

Slajd 16 — Granice rozwiązania

Cel: Opisz granice jako warunki użytkowania.

Powiedz:

> Aplikacja zarządza lokalnym urządzeniem, więc nie zastępuje centralnego systemu obsługi floty. Pełna ochrona wymaga przygotowania roli Device Owner i zastosowania polityki. Zachowanie zależy także od wersji Androida i modyfikacji producenta. Pulpit oraz dane działają lokalnie, natomiast strona WWW potrzebuje dostępności swojego serwera i odpowiedniej łączności.

Wersja krótka:

> Rozwiązanie jest lokalne, wymaga przygotowanego Androida, a działanie WebView zależy od dostępności treści.

Przejście: W tych granicach określono dziesięć wymagań funkcjonalnych.

Profesor pyta: Czy wszystko będzie działało bez internetu?

Odpowiedź: Lokalny pulpit, konfiguracja i mechanizm polityki nie wymagają centralnego serwera. Nie gwarantuję pracy strony WWW bez dostępu do jej treści; projekt nie ma własnego systemu synchronizacji offline tej strony.

Slajd 17 — Dziesięć wymagań funkcjonalnych

Cel: Zgrupuj wymagania, zamiast czytać dziesięć wierszy.

Powiedz:

> Wymagania można podzielić na trzy grupy. Pierwsza obejmuje pulpit, aplikacje i stronę. Druga dotyczy administracji: PIN-u, polityk i uprawnień. Trzecia to utrzymanie stanu, ustawienia ekranu, audyt oraz kontrolowane wyjście. Później pokażę, które elementy kodu odpowiadają każdemu wymaganiu.

Wersja krótka:

> Wymagania obejmują funkcje użytkownika, kontrolę administratora i utrzymanie środowiska po zmianach oraz restartach.

Przejście: Oprócz funkcji potrzebne są kryteria jakości działania.

Profesor pyta: Jak stwierdzić, że wymaganie zostało spełnione?

Odpowiedź: Najpierw wskazuję implementację, a następnie definiuję scenariusz akceptacyjny i oczekiwany wynik. Dla ochrony systemowej konieczny jest odczyt rzeczywistego stanu urządzenia.

Slajd 18 — Jakość środowiska pracy

Cel: Rozróżnij wymagania jakościowe i udowodnione wyniki.

Powiedz:

> Wymagania niefunkcjonalne dotyczą zgodności platformy, lokalności, odtwarzania konfiguracji, kontroli dostępu i utrzymywalności. Szczególnie ważna jest spójność: administrator ma rozróżniać konfigurację zapisaną i zastosowaną. Część tych właściwości widać w strukturze kodu. Ich zachowanie w warunkach awarii i na różnych urządzeniach wymaga osobnych prób.

Wersja krótka:

> Istotne są zgodność, lokalna praca, odtwarzanie i czytelny stan konfiguracji. Jakość działania trzeba ocenić testami.

Przejście: Do realizacji tych wymagań dobrano następujący stos technologiczny.

Profesor pyta: Czy podano limit czasu uruchomienia aplikacji?

Odpowiedź: Prezentacja nie zawiera potwierdzonego wyniku ani konkretnego progu akceptacji czasu startu. Trzeba zdefiniować sposób pomiaru i oczekiwaną wartość przed pilotażem.

Slajd 19 — Stos technologiczny

Cel: Powiąż narzędzie z zadaniem, które wykonuje.

Powiedz:

> Kotlin służy do implementacji logiki. Compose i Material 3 tworzą interfejs, a Room zapewnia warstwę nad lokalnym SQLite. Korutyny i Flow obsługują zadania oraz obserwowanie zmian. Najważniejszą integrację systemową zapewnia DevicePolicyManager, a Keystore chroni klucz poświadczenia PIN-u. Gradle buduje APK, natomiast ADB i instalatory pomagają przygotować urządzenie. SDK 36 to poziom kompilacji; minimalny poziom instalacji wynosi 34.

Wersja krótka:

> Kotlin i Compose tworzą aplikację, Room przechowuje stan, a DevicePolicyManager realizuje kontrolę urządzenia. Gradle buduje, ADB instaluje.

Przejście: Zobaczmy teraz, jak te technologie łączą się w architekturze aplikacji.

Profesor pyta: Dlaczego compileSdk 36 nie oznacza wymogu Androida 16?

Odpowiedź: O minimalnym systemie decyduje minSdk, tutaj 34. compileSdk określa API dostępne kompilatorowi, a targetSdk docelowy poziom zachowań platformy. To trzy odrębne ustawienia.

Slajd 20 — PolicyEngine łączy interfejs z systemem

Cel: Omów centralną odpowiedzialność silnika.

Powiedz:

> Centralnym elementem operacji jest PolicyEngine. Otrzymuje żądania dotyczące zapisu, zastosowania i uruchamiania sesji. Z jednej strony czyta lub aktualizuje lokalny stan, z drugiej wywołuje usługi Androida. Dzięki temu sama kontrolka w interfejsie nie jest mechanizmem blokady. Ograniczenia wynikają z polityki systemowej, a UI prezentuje ich stan i udostępnia dozwolone działania.

Wersja krótka:

> PolicyEngine przekłada działania administratora na lokalny zapis i operacje systemowe Androida.

Przejście: Pozostałe części mają własne, bardziej szczegółowe odpowiedzialności.

Profesor pyta: Czy to ścisła Clean Architecture?

Odpowiedź: To praktyczny podział odpowiedzialności z elementami MVVM. Ekran sięga także do usług aplikacji, więc nie przedstawiam go jako pełnej izolacji warstw według formalnego wzorca Clean Architecture.

Slajd 21 — Podział odpowiedzialności

Cel: Wyjaśnij, co oznaczają warstwy na slajdzie.

Powiedz:

> Interfejs odpowiada za wyświetlanie i wprowadzanie danych. ViewModel udostępnia obserwowany stan oraz obsługuje operacje ekranu. Silnik i walidatory realizują reguły działania. Magazyny utrwalają politykę, PIN oraz ustawienia strony. Ten podział ułatwia znalezienie miejsca zmiany: innego przy poprawce wyglądu, innego przy zmianie polityki urządzenia.

Wersja krótka:

> Oddzielono prezentację, stan, logikę i dane, dzięki czemu każda część ma określone zadanie.

Przejście: W silniku można wskazać cztery podstawowe operacje.

Profesor pyta: Ile modułów Gradle ma projekt?

Odpowiedź: Jeden moduł aplikacji, `:app`. Nazwy „moduł” i „warstwa” na tych slajdach opisują podział logiczny, a nie osobne moduły budowania dla każdego pliku.

Slajd 22 — Cztery operacje polityki

Cel: Odróżnij zapis, zastosowanie, uruchomienie i serwis.

Powiedz:

> `save` utrwala konfigurację administratora. `apply` próbuje zastosować ją w Androidzie i potwierdzić stan ochrony. `launch` uruchamia wskazany dozwolony program, a `serviceExit` zdejmuje ograniczenia w trybie serwisowym. Są to lokalne metody Kotlinu. Warunki w tabeli są skrótem: zastosowanie wymaga także między innymi braku aktywnej sesji i dostępnej nakładki.

Wersja krótka:

> Zapis, zastosowanie, start programu i serwis to osobne operacje, z osobnymi warunkami wejścia.

Przejście: Żeby rozdzielić ich skutki, potrzebny jest odpowiedni model danych.

Profesor pyta: Czy serviceExit odbiera aplikacji rolę Device Owner?

Odpowiedź: Nie. Wyłącza określone ograniczenia i zamiar odtwarzania ochrony. Rola właściciela, PIN i zapisane ustawienia pozostają; nie jest to pełne wycofanie urządzenia z zarządzania.

Slajd 23 — Dwie encje i ustawienia lokalne

Cel: Rozróżnij bazę polityki, dziennik i magazyny ustawień.

Powiedz:

> Room ma dwie encje. PolicyRecord przechowuje konfigurację zapisaną i ostatnio zastosowaną, a AuditEvent czas, typ i szczegóły zdarzenia. Lista reguł jest zapisana jako JSON. Drobniejsze ustawienia strony i odtwarzania trafiają do SharedPreferences. PinStore przechowuje zaszyfrowane poświadczenie w preferences i korzysta z klucza Android Keystore. Na slajdzie nie ma relacji klucza obcego między tymi encjami.

Wersja krótka:

> PolicyRecord przechowuje politykę, AuditEvent dziennik. Pozostałe ustawienia są w preferences, a klucz poświadczenia w Keystore.

Przejście: Przyjrzyjmy się teraz formatowi pojedynczej listy reguł.

Profesor pyta: Czy PIN jest trzecią tabelą Room?

Odpowiedź: Nie. PinStore ma osobny magazyn preferences i klucz Keystore. Dwie encje Room dotyczą polityki i

audytu.

Slajd 24 — Konfiguracja jest danymi

Cel: Wytłumacz JSON na prostym przykładzie.

Powiedz:

> W przykładzie każdy wpis zawiera nazwę pakietu i pole `single`. Wartość true oznacza sesję jednej aplikacji, false uruchomienie w trybie wielu aplikacji. Nazwy pakietów na slajdzie są fikcyjne. Przed przyjęciem konfiguracji aplikacja sprawdza dostępność pakietów i duplikaty. Niezależnie waliduje adresy strony. Sam poprawny JSON nie oznacza jeszcze poprawnej ani zastosowanej polityki.

Wersja krótka:

> Reguła wskazuje pakiet i tryb sesji. Konfiguracja jest walidowana, zapisywana i dopiero później stosowana.

Przejście: Taki podział widać w całej ścieżce pracy administratora.

Profesor pyta: Dlaczego w JSON-ie nie ma ikony i nazwy aplikacji?

Odpowiedź: Pakiet jest identyfikatorem reguły. Etykietę i ikonę aplikacja odczytuje z PackageManager na urządzeniu, więc nie musi dublować ich w dokumencie konfiguracji.

Slajd 25 — Zmiana polityki krok po kroku

Cel: Pokaż świadome rozdzielenie decyzji administratora i skutku systemowego.

Powiedz:

> Administrator najpierw potwierdza tożsamość PIN-em, wybiera komponenty i zapisuje nową rewizję. Następnie osobno stosuje konfigurację. Dzięki temu można przygotować zmianę bez natychmiastowej zmiany aktywnego środowiska. Błędna konfiguracja jest odrzucana. Dla PIN-u wprowadzono ograniczanie kolejnych prób: narastające opóźnienie zaczyna się od piątej nieudanej weryfikacji.

Wersja krótka:

> Ścieżka administratora to PIN, wybór, zapis i zastosowanie. Zapisana zmiana może jeszcze czekać na aktywację.

Przejście: Najważniejszy z tych kroków, zastosowanie, ma własną sekwencję kontroli.

Profesor pyta: Czy pierwszy błędny PIN blokuje dostęp na dłuższy czas?

Odpowiedź: Nie. Pierwsze cztery niepowodzenia nie dodają opóźnienia. Po piątym jest ono równe 30 sekund, potem rośnie i osiąga limit 15 minut.

Slajd 26 — Zapis nie uruchamia ochrony

Cel: Objaśnij kolejność od kontroli do potwierdzenia sukcesu.

Powiedz:

> `apply` sprawdza autoryzację i przygotowanie urządzenia, następnie ustawia listę pakietów oraz funkcje Lock Task. Aplikacja czeka na systemowy stan LOCKED i potwierdzenie gotowości nakładki. Dopiero po tych kontrolach zapisuje zastosowaną rewizję oraz aktywną politykę. Jeśli zmiana się nie powiedzie, kod podejmuje próbę przywrócenia poprzedniego stanu. Nie jest to jedna atomowa transakcja obejmująca Androida i bazę.

Wersja krótka:

> Najpierw zmiana i kontrola stanu Androida, potem zapis sukcesu. Samo utrwalenie listy nie aktywuje ochrony.

Przejście: Po zastosowaniu polityki użytkownik może uruchamiać przygotowane komponenty.

Profesor pyta: Czy błąd zawsze całkowicie cofa wszystkie zmiany?

Odpowiedź: Kod ma ścieżkę kompensacji w apply, ale także ona może napotkać błąd. Dlatego trzeba sprawdzić zapisany komunikat, audyt i rzeczywisty stan DPM, zamiast zakładać gwarantowane wycofanie.

Slajd 27 — Uruchomienie dozwolonego komponentu

Cel: Opisz przebieg w chronionym trybie i jego granice.

Powiedz:

> Ten schemat pokazuje uruchomienie w chronionym środowisku. Silnik znajduje regułę dla pakietu i sprawdza przygotowanie uprawnień. Dla pojedynczej sesji dodatkowo zawęża listę oraz zapisuje znacznik sesji. Następnie otwiera program i rejestruje zdarzenie. Zwykły powrót z sesji pojedynczej wymaga PIN-u. W kodzie istnieje też uruchamianie zwykłego programu bez aktywnej ochrony, ale nie daje ono pełnej kontroli stanowiska.

Wersja krótka:

> Chronione uruchomienie wymaga dozwolonej reguły i przygotowanych uprawnień. Pojedyncza sesja zawęża środowisko i chroni powrót PIN-em.

Przejście: Po tym przebiegu przejdę do obecnego stanu implementacji.

Profesor pyta: Czy każda aplikacja zawsze wymaga aktywnego Lock Task do startu?

Odpowiedź: Nie. Dla reguły wielu aplikacji silnik dopuszcza zwykłe uruchomienie przy wyłączonej ochronie. Pojedyncza sesja wymaga pełnej ochrony, a kafelek strony również wymaga chronionego stanu.

Slajd 28 — Stanowisko 1.0.9

Cel: Oddziel numer wydania od dowodu działania na sprzęcie.

Powiedz:

> Obecny kod opisuje wersję 1.0.9 dla Androida 14 i nowszego. Są dwa tryby pracy oraz dwie encje Room w schemacie wersji trzeciej. To wydanie rozwojowe. Sam numer wersji i dostępność kodu nie oznaczają zakończenia badań ochrony na docelowym sprzęcie.

Wersja krótka:

> Dostępna jest implementacja 1.0.9. Status kodu i status prób sprzętowych pozostają odrębne.

Przejście: Pokażę teraz, gdzie te mechanizmy znajdują się w projekcie.

Profesor pyta: Co oznacza 1.0.9 w porównaniu ze schematem v3?

Odpowiedź: 1.0.9 to wersja produktu. Schemat 3 opisuje strukturę bazy Room. Dodatkowo versionCode wynosi 11, a rewizja polityki zmienia się przy zapisie konfiguracji. Te numery mają różne znaczenie.

Slajd 29 — Struktura projektu

Cel: Wskaż najważniejsze pliki i nie czytaj całego drzewa.

Powiedz:

> Manifest deklaruje komponenty aplikacji. MainActivity otwiera główny ekran, WorkspaceScreen tworzy pulpit i panel, a ViewModel udostępnia ich stan. PolicyEngine zawiera operacje polityk. Modele danych, PIN i WebView mają własne pliki. Taki podział pozwala śledzić operację od przycisku przez logikę do zapisu lub wywołania Androida.

Wersja krótka:

> Najważniejsze obszary kodu to ekran, ViewModel, silnik polityk oraz osobne części danych, PIN-u i WWW.

Przejście: Kod trzeba następnie zbudować do postaci pakietu Androida.

Profesor pyta: Gdzie zaczyna się wykonywanie programu, skoro nie ma main?

Odpowiedź: Komponenty uruchamia Android zgodnie z manifestem. Tworzy Application, a potem odpowiednią Activity, usługę lub receiver. Własny kod reaguje metodami cyklu życia.

Slajd 30 — Budowanie i instalacja

Cel: Wyjaśnij polecenie i wynik budowania.

Powiedz:

> Polecenie assembleDebug uruchamia budowanie wariantu rozwojowego przez wrapper Gradle. Wynikiem jest APK pod wskazaną ścieżką. Konfiguracja rozróżnia minimalne API 34 oraz kompilację i target 36. Następnie ADB pozwala zainstalować pakiet na urządzeniu. Do uruchomienia Gradle potrzebne są poprawnie wskazane JDK i SDK. Taki build nie jest jeszcze gotowym, produkcyjnie podpisanym wydaniem.

Wersja krótka:

> Gradle tworzy debug APK, a ADB instaluje go na urządzeniu. Budowanie i przygotowanie produkcyjnej dystrybucji to osobne zadania.

Przejście: Pierwszą częścią implementacji, którą omówię dokładniej, jest trwały zapis danych.

Profesor pyta: Czy ten kod udało się zbudować?

Odpowiedź: W dodatkowej weryfikacji z 6 listopada wykonano assembleDebug i lintDebug z wynikiem BUILD SUCCESSFUL. Lint zgłosił 28 ostrzeżeń, bez błędów. To wynik budowania i analizy statycznej, nie test działania kiosku na tablecie.

Slajd 31 — Room przechowuje politykę i audyt

Cel: Wskaż pola, które wyrażają rozdzielenie zapisu i zastosowania.

Powiedz:

> W PolicyRecord warto zauważyć pary revision i appliedRevision oraz rules i appliedRules. Pierwsze pola opisują zapis administratora, drugie ostatnie udane zastosowanie. AuditEvent utrwala zdarzenia. Migracja 1 do 2 dodaje ustawienia niewygaszania, a 2 do 3 ustawienia pasków. Dzięki jawnej ścieżce migracji aktualizacja nie musi kasować wcześniejszej bazy, ale zachowanie migracji nadal wymaga sprawdzenia testem.

Wersja krótka:

> Room rozdziela zapisane i zastosowane reguły, przechowuje audyt oraz ma migracje ustawień ekranu i pasków.

Przejście: Dane opisują zamiar, natomiast egzekwowanie wykonują polityki Androida.

Profesor pyta: Czy appliedRevision zawsze dowodzi, że ochrona działa teraz?

Odpowiedź: To zapis ostatniego potwierdzonego zastosowania. Bieżący stan trzeba odczytać także z Androida, uwzględniając Device Owner, enabled i Lock Task. Po błędzie stan lokalny i systemowy mogą się różnić.

Slajd 32 — Device Owner jest warunkiem ochrony

Cel: Rozdziel rolę zarządzania, konfigurację listy i uruchomienie blokady.

Powiedz:

> Device Owner daje aplikacji rolę potrzebną do zarządzania urządzeniem w przyjętym modelu. Widoczne wywołania ustawiają dozwolone pakiety i funkcje Lock Task. Sam tryb jest następnie uruchamiany i sprawdzany. To działanie przez oficjalne API, bez wymagania roota w tej implementacji. Zwykłe włączenie administratora urządzenia ani instalacja APK nie zastępują roli właściciela. Ważne jest również rozróżnienie stanu LOCKED od zwykłego przypięcia ekranu.

Wersja krótka:

> Device Owner umożliwia zarządzanie, a Lock Task ogranicza zadania według ustawionej listy. Sam APK tego nie uruchamia.

Przejście: Na tej podstawie aplikacja realizuje dwa sposoby pracy.

Profesor pyta: Czy bez Device Owner nie da się w ogóle zobaczyć pulpitu?

Odpowiedź: Pulpit i część zwykłego uruchamiania są dostępne. Brakuje jednak pełnej ochrony wymaganej przez pojedynczą sesję i chronioną stronę. Roli systemowej nie zastępuje sam interfejs.

Slajd 33 — Dwa tryby pracy

Cel: Pokaż różnicę zachowania przy powrocie.

Powiedz:

> W trybie wielu aplikacji użytkownik wraca do pulpitu i wybiera inny dozwolony program. W sesji jednej aplikacji środowisko zostaje zawężone, a powrót wymaga PIN-u. Decyzję przechowuje singleSession w regule konkretnego pakietu, więc różne kafelki mogą uruchamiać różne tryby. Pływające menu zapewnia dostęp do powrotu i administratora także nad innymi aplikacjami.

Wersja krótka:

> Wiele aplikacji pozwala przełączać się przez pulpit. Pojedyncza sesja wymaga PIN-u przed powrotem.

Przejście: Podobne tryby są dostępne dla wbudowanej strony internetowej.

Profesor pyta: Czy sesja pojedyncza zostawia na liście tylko jeden pakiet?

Odpowiedź: Dla aplikacji natywnej pozostawia własny pakiet Stanowiska i pakiet celu. Własny pakiet jest potrzebny do sterowania i autoryzacji. Dla strony wystarcza pakiet Stanowiska, bo WebActivity należy do tej aplikacji.

Slajd 34 — WebView ogranicza nawigację

Cel: Zdefiniuj origin i jasno nazwij zakres ograniczenia.

Powiedz:

> WebView wyświetla stronę wewnątrz aplikacji. Lista dozwolonych adresów jest porównywana jako origins, czyli schemat, host i port. Dla przykładu inna ścieżka w tym samym origin jest dozwolona, ale subdomena lub inny port wymagają dodatkowego wpisu. Administrator ustawia też JavaScript. Jest to kontrola nawigacji; nie przedstawiam tej listy jako zapory całego ruchu sieciowego urządzenia.

Wersja krótka:

> Lista originów kontroluje nawigację strony według schematu, hosta i portu. Nie jest globalnym filtrem całej sieci.

Przejście: Drugim ważnym obszarem kontroli jest dostęp administratora przez PIN.

Profesor pyta: Czy wpisanie example.org automatycznie dopuszcza wszystkie subdomeny?

Odpowiedź: Nie. Host jest porównywany dokładnie po normalizacji. Potrzebny jest pełny adres HTTP lub HTTPS, a dodatkowy host, np. logowania, trzeba wpisać osobno. Ścieżka nie zawęża originu.

Slajd 35 — PIN chroni dostęp administratora

Cel: Wyjaśnij role kolejnych mechanizmów kryptograficznych.

Powiedz:

> PIN nie jest zapisywany jako zwykły tekst. Z losowej soli i PIN-u wyprowadzana jest pochodna PBKDF2-HMAC-SHA256 z sześciuset tysiącami iteracji. Sól z pochodną jest dodatkowo szyfrowana AES-GCM, a kluczem zarządza Android Keystore. Kolejne błędne próby wywołują opóźnienia. Przy przygotowaniu trzeba zmienić domyślne 1111. Te mechanizmy podnoszą koszt zgadywania i chronią zapis, ale nie zastępują pełnej oceny bezpieczeństwa urządzenia.

Wersja krótka:

> PIN ma sól, pochodną PBKDF2, szyfrowany zapis i ograniczanie prób. Domyślny kod trzeba zmienić przed udostępnieniem stanowiska.

Przejście: Oprócz PIN-u sprawdzana jest poprawność danych konfiguracyjnych.

Profesor pyta: Czy PBKDF2 szyfruje PIN i czy klucz zawsze jest w StrongBox?

Odpowiedź: PBKDF2 wyprowadza pochodną, nie jest szyfrowaniem odwracalnym. Szyfrowanie zapisu realizuje AES-GCM. Kod używa Android Keystore, ale nie wymusza StrongBox ani nie potwierdza jego obecności na danym urządzeniu.

Slajd 36 — Błędna konfiguracja jest odrzucana

Cel: Przypisz każdemu warunkowi sens praktyczny.

Powiedz:

> Walidator odrzuca pustą listę, jeśli nie ma włączonej strony, a także duplikaty i nieobecne aplikacje. Nie pozwala ręcznie dodać własnego pakietu, ponieważ silnik dodaje go automatycznie. Osobna walidacja adresów akceptuje tylko poprawne origins HTTP lub HTTPS bez danych logowania w URL. Dzięki temu błędy wejścia mogą zostać wykryte przed próbą zastosowania konfiguracji w systemie.

Wersja krótka:

> Sprawdzane są pakiety, duplikaty, sens listy i poprawność URL. Własny pakiet dodaje silnik, a nie administrator.

Przejście: Poprawny zapis nadal trzeba odróżnić od polityki faktycznie zastosowanej.

Profesor pyta: Czy walidacja jest wykonywana tylko podczas zapisu?

Odpowiedź: Nie. Istotne kontrole są powtarzane przy zastosowaniu i uruchamianiu. Między zapisaniem konfiguracji a użyciem aplikacja może zniknąć lub zmienić wymagane uprawnienia.

Slajd 37 — Konfiguracja zapisana i zastosowana

Cel: Objaśnij odtwarzanie na konkretnym przykładzie rewizji.

Powiedz:

> Załóżmy, że rewizja siódma działa na urządzeniu, a administrator zapisuje ósmą. Dopóki jej nie zastosuje, przy odtwarzaniu obowiązuje ostatnia zastosowana lista, czyli siódma. Mechanizm restore uwzględnia także znacznik aktywnej sesji. Po restarcie BootReceiver może poprosić o otwarcie pulpitu, a MainActivity próbuje odtworzyć ochronę. To działanie zależy od stanu urządzenia, dlatego wymaga próby na docelowym sprzęcie.

Wersja krótka:

> Odtwarzana jest ostatnia zastosowana konfiguracja, a nie nowy, niezastosowany zapis. Stan po restarcie wymaga sprawdzenia na urządzeniu.

Przejście: Na tej podstawie mogę podsumować funkcje obecne w aplikacji.

Profesor pyta: Czy każda udana operacja restore ma osobny wpis w audycie?

Odpowiedź: Nie ma osobnego zdarzenia RESTORE_SUCCESS. Kod rejestruje żądanie z BootReceiver, błędy odtwarzania i wybrane zdarzenia Lock Task. Ogólny napis na slajdzie o audycie nie oznacza kompletnego zapisu każdego kroku przywracania.

Slajd 38 — Zakres obecny w aplikacji

Cel: Podsumuj rezultat z perspektywy obu użytkowników.

Powiedz:

> Użytkownik ma kontrolowany pulpit, wybrane aplikacje, pojedynczą sesję i stronę internetową. Administrator ma PIN, konfigurację, polityki, uprawnienia, ustawienia ekranu i audyt. Materiały instalacyjne wspierają przygotowanie urządzenia. Jest to podsumowanie funkcji obecnych w implementacji; następny krok to pokazanie ich zachowania w konkretnych scenariuszach.

Wersja krótka:

> Implementacja obejmuje pracę użytkownika i administrację, od pulpitu po polityki, PIN oraz lokalny audyt.

Przejście: Do takiego sprawdzenia przygotowano następującą demonstrację.

Profesor pyta: Czy ta lista jest protokołem odbioru funkcji?

Odpowiedź: Nie. Jest zestawieniem możliwości kodu. Protokół powinien dodatkowo zawierać warunki próby, urządzenie, wynik i ocenę każdego kryterium.

Slajd 39 — Demonstracja krok po kroku

Cel: Zapowiedz pokaz i nazwij obserwacje.

Powiedz:

> Demonstrację zaczynam od roli Device Owner i konfiguracji administratora. Następnie zapisuję i stosuję politykę, sprawdzam listę aplikacji oraz powrót z pojedynczej sesji. Dla WebView sprawdzam przejście do obcego originu. Na końcu oceniam restart i wyjście serwisowe. W każdym kroku liczy się wynik zaobserwowany na urządzeniu; sama obecność kroku na tej liście nie oznacza jeszcze udanego testu.

Wersja krótka:

> Pokaz obejmuje przygotowanie, zastosowanie polityki, kontrolę uruchamiania, PIN, WebView i odtworzenie po restarcie.

Przejście: Po demonstracji wrócę do mapy kodu odpowiadającego za te zachowania.

Profesor pyta: Jak wykonać próbę aplikacji niedozwolonej, jeśli nie ma jej kafelka?

Odpowiedź: Brak kafelka sprawdza tylko zachowanie interfejsu. Próbę ograniczeń systemowych trzeba wcześniej zaprojektować dla urządzenia, np. przez kontrolowany Intent do pakietu spoza listy. Wymaga to obserwacji systemu, a nie samego zdjęcia pulpitu.

Slajd 40 — Mapa modułów

Cel: Połącz zachowanie z miejscem implementacji.

Powiedz:

> Tabela wiąże elementy pokazu z odpowiedzialnościami w kodzie. Interfejs i ViewModel obsługują pracę ekranu, PolicyEngine wykonuje operacje polityki, a baza przechowuje stan. PIN, WebView, uprawnienia,

> restart i nakładka mają własne części. Nie będę odczytywał wszystkich nazw; ważne jest, że dla każdego zachowania można wskazać konkretny punkt implementacji.

Wersja krótka:

> Każdemu zachowaniu z pokazu odpowiada konkretna część kodu: interfejs, silnik, dane albo obsługa systemu.

Przejście: Taką mapę można następnie powiązać z wymaganiami funkcjonalnymi.

Profesor pyta: Czy autoryzacja administratora jest przechowywana w ViewModelu?

Odpowiedź: Właściwy termin autoryzacji adminUntil znajduje się w WorkspaceApp, a poświadczenie w PinStore. ViewModel i ekran korzystają z usług oraz prezentują stan; wiersz tabeli jest skrótem odpowiedzialności.

Slajd 41 — Wymagania mają pokrycie w kodzie

Cel: Wyjaśnij, co ta tabela potwierdza.

Powiedz:

> Każdemu wymaganiu przypisałem element implementacji. Na przykład listę opisują AppRule i PolicyRecord, a odtwarzanie realizują restore i BootReceiver. Takie powiązanie pomaga ocenić kompletność zakresu i znaleźć miejsce testu. Oznaczenie „w kodzie” potwierdza obecność rozwiązania, natomiast nie zastępuje funkcjonalnego odbioru na urządzeniu.

Wersja krótka:

> Tabela pokazuje śledzenie wymagań do kodu. Obecność implementacji to pierwszy etap, a wyniki prób są osobnym dowodem.

Przejście: Następnie wskażę pomiary potrzebne do oceny działania.

Profesor pyta: Czego brakuje do pełnej macierzy weryfikacji wymagań?

Odpowiedź: Identyfikatora testu, warunków i urządzenia, oczekiwanego wyniku, wyniku rzeczywistego oraz oceny. Samo wskazanie klasy nie potwierdza spełnienia całego wymagania.

Slajd 42 — Trzy pomiary do wykonania

Cel: Określ badane wielkości, nie wymyślaj wyników.

Powiedz:

> Plan obejmuje czas otwarcia pulpitu, czas zastosowania polityki i zachowanie po restarcie. Dla każdego pomiaru trzeba ustalić jednoznaczny początek i koniec oraz podać model i wersję systemu. Przykładowo zastosowanie liczę do potwierdzenia LOCKED i ukończenia operacji, a nie do samego naciśnięcia przycisku. Slajd nie zawiera jeszcze zebranych wyników tych pomiarów.

Wersja krótka:

> Do zbadania pozostają start, zastosowanie polityki i restart, według jawnie określonych warunków pomiaru.

Przejście: Na tej podstawie można uczciwie określić obecny stan weryfikacji.

Profesor pyta: Jaką średnią szybkość działania uzyskano?

Odpowiedź: Nie ma zebranej serii takich pomiarów. Nie podaję więc średniej. Najpierw trzeba określić liczbę prób, stan początkowy i sposób pomiaru, a potem zebrać wyniki.

Slajd 43 — Kod i dokumentacja to pierwszy etap

Cel: Wytłumacz znaczenie dwóch statusów w tabeli.

Powiedz:

> Tabela rozróżnia właściwości widoczne w konfiguracji i kodzie od zachowań wymagających prób. Można odczytać minimalne API, model lokalnych danych i podział rewizji. Odtwarzanie oraz skuteczność ochrony trzeba ocenić na urządzeniu. Dodatkowe budowanie i sprawdzenia logiki z dokumentacji poszerzają weryfikację, ale nie zmieniają automatycznie tych dwóch obszarów na przetestowane sprzętowo.

Wersja krótka:

> Stan „w kodzie” oznacza implementację, a „do prób” potrzebę potwierdzenia jej zachowania na sprzęcie.

Przejście: To samo rozróżnienie stosuję do pięciu tez pracy.

Profesor pyta: Czy 62 sprawdzenia z dokumentacji oznaczają 62 testy kiosku?

Odpowiedź: Nie. Obejmowały wybrane skompilowane funkcje pomocnicze, np. walidację, opóźnienia PIN-u i normalizację adresów. Nie uruchamiały DevicePolicyManager, nakładki ani całej sesji na fizycznym Androidzie.

Slajd 44 — Jak zweryfikować pięć tez

Cel: Dopasuj metodę sprawdzenia do każdej tezy.

Powiedz:

> Skuteczność ograniczeń sprawdzam próbami uruchomienia niedozwolonych komponentów i opuszczenia środowiska. Elastyczność oceniam przez konfigurację różnych scenariuszy. Użycie mechanizmów systemowych potwierdzam odczytem roli i stanu Lock Task. Teza o mniejszym nakładzie pracy wymaga porównania czasu przygotowania takiego samego stanowiska dwiema metodami. Sama wygoda formularza nie dowodzi jeszcze oszczędności czasu.

Wersja krótka:

> Każda teza ma własną metodę weryfikacji: próby ograniczeń, różne konfiguracje, odczyty systemu i porównanie czasu administracji.

Przejście: Przed badaniami trzeba poprawnie zainstalować i przygotować aplikację.

Profesor pyta: Jak zapewnić uczciwe porównanie czasu administracji?

Odpowiedź: Obie metody muszą prowadzić do tego samego stanu końcowego na porównywalnym urządzeniu. Trzeba oddzielić jednorazowe przygotowanie od późniejszych zmian i opisać doświadczenie osoby wykonującej zadanie.

Slajd 45 — Instalacja i przygotowanie urządzenia

Cel: Oddziel instalację APK, rolę właściciela i aktywację polityki.

Powiedz:

> ADB najpierw potwierdza dostęp do urządzenia. Instalacja z flagą r pozwala aktualizować zgodny pakiet z

> zachowaniem danych, a t dopuszcza testowy APK. Osobne polecenie nadaje Device Owner, jeśli Android akceptuje stan urządzenia. Potem trzeba zmienić PIN, przygotować uprawnienia i nakładkę, zapisać oraz zastosować politykę. Instalatory Bash i PowerShell automatyzują część tej procedury, ale nie obchodzą warunków systemu.

Wersja krótka:

> Instalacja APK, nadanie Device Owner i zastosowanie polityki to trzy różne etapy przygotowania urządzenia.

Przejście: Na końcu pozostają kierunki rozwoju po lokalnej implementacji.

Profesor pyta: Dlaczego aktualizacja z flagą r może zostać odrzucona?

Odpowiedź: Android sprawdza m.in. zgodność pakietu i podpisu. Nowy lokalny APK debug może mieć inny klucz niż wcześniejsza instalacja. Flaga r nie omija tej kontroli i nie jest gwarancją udanej aktualizacji.

Slajd 46 — Kolejne etapy rozwoju

Cel: Pokaż priorytety dalszych prac.

Powiedz:

> Najbliższy krok to pilotaż, próby migracji i odtwarzania oraz ocena użyteczności. Kolejne usprawnienia mogą obejmować import konfiguracji, szablony i procedurę odzyskiwania dostępu. Przed szerszą dystrybucją potrzebne są także podpis produkcyjny i przemyślany provisioning. Integracja z panelem centralnym przez API jest dalszym rozszerzeniem, którego obecna aplikacja lokalna nie implementuje.

Wersja krótka:

> Priorytetem są próby i przygotowanie dystrybucji. Import, QR oraz panel centralny to kolejne etapy rozwoju.

Przejście: Podsumuję teraz rezultat i jego najważniejsze znaczenie.

Profesor pyta: Które z tych rozszerzeń już działają?

Odpowiedź: Wymienione w kolumnie „kolejny etap” są propozycjami. Nie przedstawiam importu i eksportu,

provisioningu QR ani centralnego API jako obecnych funkcji.

Slajd 47 — Kontrolowane środowisko i lokalna konfiguracja

Cel: Zamknij prezentację konkretnym rezultatem i następnym krokiem.

Powiedz:

> Rezultatem jest aplikacja łącząca kontrolowany pulpit, lokalną konfigurację i polityki Androida. Najważniejsze rozdzielenia to zapis i zastosowanie oraz zwykła praca i autoryzowana administracja. Implementacja daje podstawę do przygotowania różnych stanowisk, natomiast potwierdzenie skuteczności tez wymaga pilotażu i pomiarów na docelowym sprzęcie. To jest kolejny krok rozwoju projektu. Dziękuję za uwagę i jestem gotowy na pytania.

Wersja krótka:

> Powstała implementacja lokalnego stanowiska Android. Kolejny krok to pilotaż i pomiary potwierdzające jego działanie. Dziękuję za uwagę.

Przejście: Zatrzymaj slajd i poczekaj na pytania. Nie przełączaj już prezentacji.

Profesor pyta: Jaki jest najważniejszy wkład projektu?

Odpowiedź: Połączenie w jednej lokalnej aplikacji konfiguracji komponentów, dwóch trybów pracy, systemowych polityk, autoryzacji i trwałego stanu. Praktycznym celem jest przygotowanie urządzenia do określonego zadania w granicach mechanizmów Androida.

Demonstracja przy slajdzie 39

Wybierz sposób pokazania przed wystąpieniem

Jeśli tablet nie jest wcześniej przygotowany i sprawdzony, potraktuj slajd jako omówienie **procedury weryfikacji**. Pełne nadawanie Device Owner, pierwsze logowanie w zewnętrznej aplikacji czy dobieranie adresów SSO podczas prezentacji utrudnia ocenę właściwego mechanizmu. Na pokaz przygotuj dwa znane pakiety i kontrolowaną stronę bez rzeczywistych danych użytkowników.

Przy gotowym urządzeniu możesz powiedzieć: „Pokażę teraz kilka zachowań na tym konkretnym urządzeniu. Będę odróżniał oczekiwany efekt od wyniku, który rzeczywiście zaobserwujemy”.

Krok	Co robisz	Co mówisz i obserwujesz
Rola	Odczyt właściciela i stanu w panelu	„Najpierw potwierdzam przygotowanie Androida. Sama ikona aplikacji nie dowodzi roli właściciela”.
Administrator	Wejście PIN-em; kod początkowy zmieniony przed udostępnieniem	„To lokalna autoryzacja dostępu do ustawień. Nie jest to konto w centralnym systemie”.
Zapis	Zmiana listy i zapis konfiguracji	„Wzrosła rewizja zapisana. Aktywna lista nie musi jeszcze się zmienić”.
Zastosowanie	Naciśnięcie przycisku i kontrola wyniku	„Teraz porównuję rewizje oraz potwierdzenie systemowego Lock Task”.
Wieloaplikacyjność	Otwarcie A, powrót przez dom, otwarcie B	„Powrót do przygotowanego pulpitu jest dozwolony; ochrona pozostaje aktywna”.
Niedozwolony pakiet	Wcześniej przygotowana, kontrolowana próba uruchomienia poza listą	„To sprawdza ograniczenie systemu. Sam brak kafelka byłby jedynie sprawdzeniem interfejsu”.
Pojedyncza sesja	Uruchomienie odpowiedniej reguły i przycisk kłódki	„Powrót wymaga PIN-u. Po autoryzowanym zakończeniu wraca lista wielu aplikacji”.
WWW	Dozwolony adres i link do obcego originu	„Sprawdzam nawigację. Ten pokaz nie jest pomiarem całego ruchu sieciowego”.
Restart	Restart po przywróceniu chronionej konfiguracji	„Po odblokowaniu ekranu sprawdzam, czy wróciły zastosowana lista i stan ochrony”.
Serwis	Zakończenie sesji, autoryzacja, wyjście serwisowe	„Sprawdzam zniesienie ograniczeń. Device Owner pozostaje nadany”.

Pełny pokaz obejmuje oczekiwanie na system i interakcje, więc nie doliczaj go jako jednego krótkiego komentarza do slajdu. Przy małej ilości czasu wybierz zapis/zastosowanie, jeden powrót z sesji i WWW; pozostałe próby omów jako plan. Nie przedstawiaj pominiętych kroków jako zaliczonych.

Gdy pokaz nie powiedzie się

Jeśli brakuje Device Owner: „Urządzenie nie spełnia warunku rozpoczęcia pełnej ochrony. Nie będę nadawał tej roli podczas wystąpienia. Pokażę sekwencję operacji i wrócę do przygotowania sprzętu przed właściwą próbą”.

Jeśli strona się nie otwiera: „Najpierw rozróżniam błąd sieci, certyfikatu i listy originów. Brak strony nie dowodzi, że polityka lokalnego pulpitu przestała działać”.

Jeśli zastosowanie zgłasza błąd: „To wynik tej próby. Sprawdzę komunikat, zapis błędu i stan Androida. Samo kliknięcie przycisku nie jest potwierdzeniem aktywacji”.

Jeśli nie masz urządzenia: „Ten slajd przedstawia przygotowany scenariusz, a nie wykonany dzisiaj test. Działanie systemowych ograniczeń pozostaje do potwierdzenia na sprzęcie”. Następnie przejdź do slajdu 40.

Doprecyzowania do trudniejszych slajdów

Slajd	Jak rozumieć skrót na slajdzie	Sformułowanie do zapamiętania
5 i 44	Tezy nie są jeszcze wynikami eksperymentu	„Implementacja uzasadnia sposób realizacji; skuteczność weryfikuję próbami”.
9	Lokalizacja pilotażu nie oznacza produkcyjnego wdrożenia	„To proponowane środowisko testowe”.
13	Kwota nie opisuje pełnego kosztu projektu	„Nie obejmuje urządzenia, pracy i utrzymania”.
15	Oceny ryzyka są jakościowe	„Nie są częstością awarii z próby statystycznej”.
21 i 40	Moduły na slajdzie to odpowiedzialności w jednym `:app`	„Podział logiczny, nie dziesięć modułów Gradle”.
22 i 27	Warunki uruchomienia są skrócone	„Mówię o chronionym przepływie; zwykły start wielu aplikacji jest możliwy bez pełnej ochrony”.
25	Opóźnienie PIN-u nie zaczyna się od pierwszego błędu	„Od piątej nieudanej próby”.
26	Kompensacja nie jest transakcją DPM i Room	„Kod próbuje odtworzyć poprzedni stan i zapisuje błędy”.
32	Device Owner umożliwia zarządzanie, ale nie wystarcza do ochrony	„Rolę trzeba uzupełnić zastosowaną polityką i stanem LOCKED”.
34	Origin nie jest samą domeną i nie filtruje wszystkiego	„Schemat, host, port; kontrola nawigacji”.
35	PBKDF2, AES-GCM i Keystore mają różne zadania	„Pochodna, szyfrowanie zapisu, zarządzanie kluczem”.
37	restore bierze zastosowane reguły; audyt nie zawiera osobnego sukcesu restore	„Ostatnia zastosowana konfiguracja, z kontrolą bieżącego systemu”.
39	Ukryty kafelek nie dowodzi systemowej blokady innego programu	„Potrzebna jest osobna próba uruchomienia spoza listy”.
41–43	Kod i test to różne rodzaje dowodów	„Jest implementacja; określona próba potwierdza określone zachowanie”.
45	Serwis nie usuwa zarządzania ani wcześniejszych decyzji uprawnień	„Ograniczenia stanowiska można wyłączyć, pozostawiając Device Owner”.
46	Roadmapa nie opisuje już dostępnych funkcji	„To propozycje kolejnych etapów”.

Ostatnia powtórka przed wystąpieniem

Wątki, do których możesz wrócić po pytaniu

1. **Device Owner → polityka → LOCKED.** Rola, konfiguracja i potwierdzone działanie to osobne elementy.
2. **save → apply.** Pierwsze zmienia zapis, drugie próbuje zmienić system i utrwalić wynik.
3. **revision → appliedRevision.** Potrafisz wyjaśnić przykład z rewizją 7 i 8.
4. **Wiele → jedna aplikacja.** Rozróżniasz powrót do pulpitu, koniec sesji, zamknięcie panelu i wyjście serwisowe.
5. **PIN.** PBKDF2 tworzy pochodną, AES-GCM szyfruje zapis, Keystore zarządza kluczem; od piątej porażki rośnie

opóźnienie.

6. **Origin.** Znasz przykład innej ścieżki, subdomeny i portu oraz granicę kontroli nawigacji.
7. **Implementacja → weryfikacja.** Umiesz nazwać, co faktycznie sprawdzono i co wymaga sprzętu.

Jak ćwiczyć

Najpierw przejdź całą prezentację z wypowiedziami „Powiedz", bez czytania pytań profesora. Zmierz czas. Jeśli przekracza limit, skracaj przede wszystkim powtarzające się slajdy: 6/20/21/40, 17/38/41 oraz 18/42/43. Zachowaj różnicę zapisu i zastosowania, wyjaśnienie Device Owner, dwa tryby, model danych, PIN i granice WebView.

Przy krótszej wersji nie pomijaj slajdów bez wyjaśnienia: użyj zdania „Wersja krótka" i przejdź dalej. Jeśli limit jest bardzo mały, ewentualny wybór mniejszej liczby slajdów ustal wcześniej. Ten scenariusz zachowuje wszystkie 47 w oryginalnej kolejności.

W drugiej próbie odpowiadaj na pytania po jednym zdaniu, a szczegóły dopowiadaj dopiero na prośbę. Gdy nie znasz wyniku, powiedz: „Tego nie potwierdziłem pomiarem. Mogę wskazać implementację i sposób sprawdzenia". Nie zastępuj wyniku przypuszczeniem.

Przygotowanie na jutro

Na dziś wybierz jedną próbę trwającą około 45–60 minut. Przez pierwsze 10 minut powtórz siedem głównych rozróżnień powyżej. Następnie przejdź slajdy z wypowiedziami podstawowymi i stoperem, bez zatrzymywania się po każdym błędzie. Po tej próbie zaznacz najwyżej pięć miejsc, w których zabrakło Ci wyjaśnienia. Przez kolejne 10–15 minut popraw właśnie te fragmenty i odpowiedz na pytania do slajdów 22, 26, 31–35, 37 i 43. Na końcu powiedz otwarcie i zakończenie bez czytania.

Jutro przed wystąpieniem wystarczy krótka powtórka ze ściągi: kolejność tematów, przykład rewizji 7/8, różnica Device Owner/Lock Task, pochodna/szyfrowanie oraz wynik kodu/wynik próby. Sprawdź dostęp do prezentacji i notatek. Jeśli planujesz demonstrację, przygotuj urządzenie przed wejściem; jeśli go nie ma, użyj wariantu omówienia procedury ze slajdu 39.

Nie ucz się całego modułu słowo w słowo. Zapamiętaj sens, pierwsze zdanie każdego ważnego slajdu i drogę do następnego tematu. Gotowe wypowiedzi mają być podporą do ćwiczenia.

Nazwy, które warto wypowiedzieć na głos

Zrób krótką próbę z nazwami Device Owner, Lock Task, PolicyEngine, ViewModel, WebView, Room i Keystore. Nie musisz odczytywać pełnych pakietów Kotlin ani całych wywołań ze slajdu. PBKDF2 możesz przedstawić jako „funkcję wyprowadzania pochodnej PIN-u", a potem dopiero podać nazwę algorytmu.

Krótkie otwarcie i zakończenie do nauczenia

Otwarcie: „Przedstawiam Stanowisko, czyli lokalnie konfigurowaną aplikację Android, która udostępnia wybrane funkcje urządzenia i korzysta z systemowych polityk do ograniczania środowiska pracy”.

Zakończenie: „Najważniejszym rezultatem jest połączenie pulpitu, trwałej konfiguracji i polityk Androida. Następnym krokiem są próby na docelowym urządzeniu, które pozwolą ocenić skuteczność rozwiązania i postawione tezy”.